BCSE302P DATABASE SYSTEMS LAB · CASE STUDY 6

DBMS Project Viva Questions

Railway Reservation and Passenger Service Management System — viva questions and answers for every query, file by file

---

Database: Oracle AI Database 26ai Free (release 23.26) · SQL and PL/SQL
Project files covered: parts/run_all.sql, 01_ddl.sql, 02_data.sql, 02b_seat.sql, 03_dml.sql, 04_queries.sql,
railway_case_study6.sql, tests/ (6 SQL suites, 2 Python scripts)
Outputs captured from the project database on 28 October 2026

Contents

How to Use This Viva Guide 4

3. File 01_ddl.sql — Schema, Constraints, Indexes, Trigger 5

Q01 · Clean-up block (Part 0) 5
Q02 · CREATE TABLE station 6
Q03 · CREATE TABLE route 7
Q04 · CREATE TABLE route_stop 7
Q05 · CREATE TABLE train 8
Q06 · CREATE TABLE coach_class 9
Q07 · CREATE TABLE coach 9
Q08 · CREATE TABLE seat 10
Q09 · CREATE TABLE train_schedule 11
Q10 · CREATE TABLE passenger 11
Q11 · CREATE TABLE reservation 12
Q12 · Unique index ux_res_seat_journey 13
Q13 · Unique index ux_res_pass_journey 14
Q14 · Supporting indexes 14
Q15 · CREATE TABLE payment 15
Q16 · CREATE TABLE cancellation 15
Q17 · Sequences 16
Q18 · Trigger trg_reservation_validate 17

4. Files 02_data.sql and 02b_seat.sql — Loading the Sample Data 19

Q19 · INSERT stations (2.1) 19
Q20 · INSERT routes and route stops (2.2) 19
Q21 · INSERT coach classes (2.3) 20
Q22 · INSERT trains (2.4) 20
Q23 · INSERT coaches (2.5) 21
Q24 · Seat generator: INSERT … SELECT with CONNECT BY (2.6 and 02b_seat.sql) 22
Q25 · INSERT train schedules (2.7) 23
Q26 · INSERT passengers (2.8) 23
Q27 · INSERT reservations (2.9) 24

5. File 03_dml.sql — Modification, Integrity Tests and Stored Procedures 25

Q28 · Generate payments with INSERT … SELECT (2.10) 25
Q29 · Generate cancellations with INSERT … SELECT (2.11) 25
Q30 · Transaction control in the scripts (COMMIT and ROLLBACK) 26
Q31 · Row counts with UNION ALL 26
Q32 · ALTER TABLE passenger ADD concession_type (3.1) 27
Q33 · ALTER TABLE station MODIFY (3.2) 27
Q34 · ALTER TABLE passenger RENAME COLUMN (3.3) 28
Q35 · ALTER TABLE train ADD CONSTRAINT (3.4) 28
Q36 · UPDATE concession from age (4.1) 29
Q37 · UPDATE train timetable (4.2) 29
Q38 · Find a PNR into a bind variable (4.3) 30
Q39 · Before-and-after check of the ticket and payment (4.3) 30
Q40 · UPDATE boarding station and recalculate fare (4.3) 31
Q41 · UPDATE payment using a scalar subquery (4.3) 31
Q42 · INSERT then DELETE a test passenger (4.4) 32
Q43 · Test 5.1 — invalid phone 33
Q44 · Test 5.2 — double seat booking 33
Q45 · Test 5.3 — seat of another train 34
Q46 · Test 5.4 — wrong direction 34
Q47 · Test 5.5 — confirmed ticket without a seat 35

Q48 · Test 5.6 — delete a parent row 35
Q49 · Test 5.7 — unknown station 36
Q50 · Procedure book_ticket (6.1) 36
Q51 · Procedure cancel_ticket (6.2) 37
Q52 · Demo reset and new journey 130 (6.3) 38
Q53 · Demo: nine bookings in a loop (6.3) 39
Q54 · Demo: cancel and auto-promote (6.4) 39
Q55 · Demo result: six-table join (6.4) 40

6. File 04_queries.sql — Views, Analysis Queries and the Capacity-Risk Module 41
Q56 · VIEW v_reservation_performance 41
Q57 · VIEW v_train_utilization 42
Q58 · VIEW v_ticket_details 42
Q59 · Querying the two summary views 43
Q60 · Q1 Train-wise reservation statistics 43
Q61 · Q2 Station-wise passenger volume 44
Q62 · Q3 Route utilization 44
Q63 · Q4 Coach occupancy 45
Q64 · Q5a Cancellation statistics by train 45
Q65 · Q5b Cancellation reasons 46
Q66 · Q6a Peak travel routes (DENSE_RANK) 46
Q67 · Q6b Top 5 origin–destination pairs 47
Q68 · Q6c Peak travel days of the week 47
Q69 · Q7 Highest and lowest utilized trains 48
Q70 · Q8 Passenger travel frequency 48
Q71 · Q9 Revenue by train with ROLLUP 49
Q72 · Q10 Routes requiring additional capacity (HAVING) 49
Q73 · A1 Ten-table join: a passenger's travel history 50
Q74 · A2 Correlated subquery: trains above their route's average 50
Q75 · A3 Passengers with repeated cancellations 51
Q76 · A4 Nested subquery: routes above average demand 51
Q77 · A5 WITH clause: stations above average traffic 52
Q78 · A6 NOT EXISTS + EXISTS: trains needing capacity 52
Q79 · A7 Correlated HAVING: above-average spenders per city 53
Q80 · A8 IN and NOT IN: loyal passengers 53
Q81 · A9 Correlated subquery on a CTE: under-used coaches 54
Q82 · CREATE TABLE risk_threshold and its rows 54
Q83 · CREATE TABLE capacity_risk_log 55
Q84 · FUNCTION fn_capacity_risk 56
Q85 · 10.1 Decision-support report 56
Q86 · PROCEDURE prc_capacity_risk_report (10.2) 57
Q87 · 10.3 What-if analysis with ROLLBACK 58

7. Runner Script and SQL*Plus Commands 59
R1 · parts/run_all.sql — the master runner 59

How to Use This Viva Guide

This document collects the viva questions for **every query** of the Railway Reservation project, in the same order and with the same numbers (Q01–Q87 and R1) as the main Query Explanation Guide.

For each query you will find:

- **How to explain this to the examiner:** a short answer you can say aloud.
- **Viva questions**, each with:
  - **Short answer:** what to say first.
  - **Detailed:** the reasoning, if the examiner asks a follow-up.
  - **Project connection:** where it appears in your project.

Tip: read the query in the main guide first, then come here and try answering each question before reading the answer.

3. File 01_ddl.sql — Schema, Constraints, Indexes, Trigger

Q01 · Clean-up block (Part 0)

HOW TO EXPLAIN THIS TO THE EXAMINER

"Part 0 makes the script re-runnable. It is an anonymous PL/SQL block that queries the data dictionary — USER_VIEWS, USER_TABLES and so on — for our project's objects and drops only the ones that exist. We need EXECUTE IMMEDIATE because DDL cannot be written directly inside PL/SQL. Tables are dropped with CASCADE CONSTRAINTS so foreign keys don't block the drop, and with PURGE so they don't go to the recycle bin. Indexes and the trigger disappear automatically with their tables."

Q1. Why can't you write `DROP TABLE station;` directly inside BEGIN…END?
Short answer: Static PL/SQL does not allow DDL.
Detailed: PL/SQL compiles static SQL only for DML, SELECT INTO, transaction control and LOCK TABLE, because DDL changes objects the code itself may depend on. DDL must be run through dynamic SQL, `EXECUTE IMMEDIATE '…'`, which is parsed at run time.
Project connection: every DROP in Q01.

Q2. Can these drops be rolled back?
Short answer: No.
Detailed: Each DROP is DDL and commits implicitly. PURGE also bypasses the recycle bin, so `FLASHBACK TABLE … TO BEFORE DROP` cannot restore them either.
Project connection: Q01, all four loops.

Q3. Why is the trigger not dropped explicitly?
Short answer: A trigger belongs to its table and is dropped with it.
Detailed: Dropping a table also drops its indexes, constraints and triggers. Views and stored procedures are independent objects; they only become INVALID, so they need their own loops.
Project connection: `trg_reservation_validate` on RESERVATION.

Q4. Is the string concatenation an SQL-injection risk?
Short answer: Not here.
Detailed: The names come from Oracle's own dictionary and are filtered by a hard-coded list; no user input reaches the string. If a name ever came from a user, it should be validated with `DBMS_ASSERT.SIMPLE_SQL_NAME` first.
Project connection: `EXECUTE IMMEDIATE 'DROP TABLE ' || t.table_name …`.

Q5. The loop drops tables while reading USER_TABLES. Why doesn't that break the loop?
Short answer: Read consistency.
Detailed: Oracle gives each query a consistent snapshot as of the moment it started. The cursor keeps returning the list that existed when it opened, even though the tables are disappearing.
Project connection: loop 2 of Q01.

Q02 · CREATE TABLE station

HOW TO EXPLAIN THIS TO THE EXAMINER

"STATION is the master table for the rail network. station_code is the primary key, so every station is uniquely identified, and a CHECK constraint forces it to uppercase so we can't get MAS and mas as two stations. All descriptive columns are NOT NULL, and platforms has a default of 1 with a CHECK limiting it to 1–30. Every other table that mentions a station uses a foreign key to this table."

Q1. What is the difference between the inline constraint on station_code and the separate CONSTRAINT line at the bottom?
Short answer: Only where they are written.
Detailed: An inline (column-level) constraint is written with the column and can refer only to that column. An out-of-line (table-level) constraint is written after the columns and can involve several columns (needed for composite keys). Both behave the same once created.
Project connection: `pk_station` is inline; `ck_station_code` is out-of-line.

Q2. What happens on `INSERT INTO station (station_code, station_name, city, state, rail_zone) VALUES ('XYZ','Test','C','S','SR')`?
Short answer: It succeeds with platforms = 1.
Detailed: platforms is omitted, so DEFAULT 1 is used; 1 satisfies NOT NULL and the BETWEEN check.
Project connection: DEFAULT on `platforms`.

Q3. Why name constraints at all?
Short answer: Readable errors and easy maintenance.
Detailed: Error messages show the constraint name, and ALTER TABLE … DROP/DISABLE CONSTRAINT needs a name. Unnamed constraints get system names like SYS_C008123 that change on every run.
Project connection: error messages in Part 5 of 03_dml.sql, e.g. CK_PASS_PHONE.

Q4. Why is the primary key a text code instead of a sequence number?
Short answer: It is a natural, stable, meaningful key.
Detailed: Station codes are already unique in the real world and never change, so they make good natural keys and make child rows self-explanatory. Surrogate numbers are better when the natural identifier can change or is long.
Project connection: compare with COACH, which uses surrogate `coach_id`.

Q03 · CREATE TABLE route

HOW TO EXPLAIN THIS TO THE EXAMINER

"ROUTE stores each corridor with a numeric key and a unique name. It has two foreign keys to STATION — origin and destination — so the same parent table plays two roles, and a CHECK makes sure the two ends differ. Because route_name is unique, our reports can safely group by it."

Q1. Can a table have two foreign keys to the same parent?
Short answer: Yes.
Detailed: Each FK is an independent constraint. Here they represent different roles (origin, destination). In queries the parent must be joined once per role using different aliases.
Project connection: `fk_route_origin`, `fk_route_dest`; `v_train_utilization` joins station as `so` and `sd`.

Q2. Why is `ck_route_ends` written at table level?
Short answer: It references two columns.
Detailed: A column-level constraint may only mention its own column. Any CHECK comparing columns must be out-of-line.
Project connection: `origin_code <> dest_code`.

Q3. What is the default delete rule of a foreign key in Oracle?
Short answer: Restrict (NO ACTION).
Detailed: Unless ON DELETE CASCADE or ON DELETE SET NULL is specified, deleting a parent row that has children fails with ORA-02292. Oracle has no ON UPDATE CASCADE.
Project connection: ROUTE → STATION has no ON DELETE clause.

Q04 · CREATE TABLE route_stop

HOW TO EXPLAIN THIS TO THE EXAMINER

"ROUTE_STOP resolves the many-to-many relationship between routes and stations. Its primary key is the composite (route_id, stop_seq), and a second unique key (route_id, station_code) stops a station appearing twice on a route. It stores each stop's distance from the origin, which the trigger uses to compute the fare and to check that the passenger travels in the right direction."

Q1. Why is this called a junction or associative table?
Short answer: It turns an M:N relationship into two 1:N relationships.
Detailed: A route has many stations and a station is on many routes. Relational tables cannot hold a list in one column (1NF), so each pairing becomes a row here with FKs to both sides.
Project connection: ROUTE 1:N ROUTE_STOP N:1 STATION.

Q2. Why both a composite PK and a composite UNIQUE?
Short answer: Two different rules.
Detailed: The PK says each position on a route holds one station; the UNIQUE says each station occupies one position on a route. Together they make the mapping one-to-one within a route.
Project connection: `pk_route_stop`, `uq_rs_station`.

Q3. How does the trigger use this table?
Short answer: Direction and distance.
Detailed: It looks up stop_seq and km_from_origin for the boarding and destination stations; if from_seq >= to_seq it raises ORA-20002, otherwise distance = to_km − from_km.
Project connection: rules R1 and R4 in Q18.

Q05 · CREATE TABLE train

HOW TO EXPLAIN THIS TO THE EXAMINER

"TRAIN holds each train service. train_no is a five-digit primary key enforced by a CHECK, train_type is limited to a fixed list, and the departure time is validated with a regular expression so only valid HH:MM times between 00:00 and 23:59 are stored. Every train references the route it runs on, and the dated journeys are kept separately in TRAIN_SCHEDULE."

Q1. Can a column have more than one constraint?
Short answer: Yes.
Detailed: train_no has a PRIMARY KEY and a CHECK; dep_time has NOT NULL and a CHECK. Every constraint is checked independently on each INSERT/UPDATE.
Project connection: train_no.

Q2. Why are TRAIN and TRAIN_SCHEDULE separate tables?
Short answer: To avoid repeating train details for every date.
Detailed: Name, type and route depend only on train_no. Putting them in the schedule table would create a dependency schedule_id → train_no → name (transitive), violating 3NF and causing update anomalies.
Project connection: TRAIN vs TRAIN_SCHEDULE.

Q3. What does `^[YN]{7}$` accept?
Short answer: Exactly seven characters, each Y or N.
Detailed: `^` and `$` anchor the start and end so nothing else may appear; `[YN]` is one character from the set; `{7}` repeats it seven times. 'YYYYYNN' passes, 'YYYYYY' (six) fails.
Project connection: ck_train_days.

Q06 · CREATE TABLE coach_class

HOW TO EXPLAIN THIS TO THE EXAMINER

"COACH_CLASS is a lookup table that keeps the business rules for each class: fare per kilometre, reservation fee and seats per coach. Keeping these as data instead of hard-coding them means a fare revision is a single UPDATE, and the trigger reads this table to compute every new fare."

Q1. What does NUMBER(5,2) mean?
Short answer: Up to 5 significant digits, 2 after the decimal point.
Detailed: Precision 5, scale 2, so the maximum is 999.99. Inserting 1000 gives ORA-01438; inserting 1.456 is rounded to 1.46.
Project connection: fare_per_km.

Q2. Why are seats_per_coach values so small (4–12)?
Short answer: For demonstration.
Detailed: The data comment says they are scaled down so that trains fill up with a few hundred tickets, making the waitlist, occupancy and capacity-risk logic visible with small data.
Project connection: 02_data.sql section 2.3.

Q3. Where is this table used?
Short answer: Trigger, seat generator, views, booking.
Detailed: The trigger reads fare_per_km and reservation_fee (R4); Q24 uses seats_per_coach to generate seats; `v_reservation_performance` sums seats_per_coach for capacity; Q4 and A9 compute coach occupancy.
Project connection: Q18, Q24, Q56, Q63, Q81.

Q07 · CREATE TABLE coach

HOW TO EXPLAIN THIS TO THE EXAMINER

"COACH is a weak entity in real life — a coach is identified by its train and its label — so we enforce UNIQUE (train_no, coach_label), but we use a surrogate coach_id as the primary key to keep the SEAT foreign key simple. Deleting a train cascades to its coaches, and the class foreign key decides the class of every seat inside the coach."

Q1. What is a weak entity?
Short answer: An entity that cannot be identified without its owner.
Detailed: Label C1 alone is meaningless; "C1 of train 12639" identifies the coach. In ER diagrams it has a partial key (coach_label) plus the owner's key.
Project connection: COACH and TRAIN.

Q2. Why does COACH carry class_code instead of SEAT?
Short answer: All seats in a coach share the class.
Detailed: Storing class on SEAT would repeat it for every seat and allow seats of one coach to disagree. That would be a transitive dependency seat_id → coach_id → class_code.
Project connection: rule R2 joins seat → coach to get the class.

Q3. What happens to seats if a coach is deleted?
Short answer: They are deleted too, unless a ticket references them.
Detailed: fk_seat_coach has ON DELETE CASCADE. But if any reservation references one of those seats, fk_res_seat (no cascade) blocks the whole delete with ORA-02292.
Project connection: cascade chain TRAIN → COACH → SEAT → (blocked by) RESERVATION.

Q08 · CREATE TABLE seat

HOW TO EXPLAIN THIS TO THE EXAMINER

"SEAT stores the physical seats of every coach, with a UNIQUE on (coach_id, seat_no). It describes inventory only — whether a seat is taken on a particular journey is decided by the RESERVATION table, which is why one seat row can be sold on many different dates."

Q1. How does the system know a seat is free on a journey?
Short answer: No reservation exists for that seat on that schedule.
Detailed: book_ticket uses `NOT EXISTS (SELECT 1 FROM reservation r WHERE r.schedule_id = p_schedule AND r.seat_id = st.seat_id)`. Cancelled tickets have seat_id NULL, so a cancelled seat becomes free automatically.
Project connection: Q50.

Q2. Why is the berth list mixing sleeper berths and chair-car seats?
Short answer: One table serves all classes.
Detailed: The CHECK lists every valid type for any class. The seat generator chooses the right subset per class (e.g. EC gets WINDOW/AISLE). A stricter design would check the type against the class, which needs a trigger or a lookup table.
Project connection: Q24.

Q3. Why ON DELETE CASCADE from COACH?
Short answer: Seats cannot exist without their coach.
Detailed: A seat is part of a coach; deleting the coach should remove its seats. Tickets still protect history, because a seat referenced by a reservation blocks the delete.
Project connection: fk_seat_coach.

Q09 · CREATE TABLE train_schedule

HOW TO EXPLAIN THIS TO THE EXAMINER

"TRAIN_SCHEDULE stores each dated run of a train. A unique constraint on (train_no, journey_date) means a train runs at most once a day, and a CHECK forces journey_date to have no time part — otherwise two rows on the same day with different times would slip past the unique constraint. Tickets reference this table, not TRAIN, because you book a specific journey."

Q1. Why is `journey_date = TRUNC(journey_date)` necessary?
Short answer: So the unique constraint really means "one per day".
Detailed: Oracle DATE stores time to the second. Without the check, rows at different times on the same day are different values and both pass `uq_sched`.
Project connection: ck_sched_date + uq_sched.

Q2. What does TRUNC do to a date?
Short answer: Removes the time (sets it to 00:00:00).
Detailed: `TRUNC(date)` truncates to the day by default; `TRUNC(date,'MM')` truncates to the first of the month. On numbers, TRUNC cuts off decimals.
Project connection: also used in `booking_date DEFAULT TRUNC(SYSDATE)` and `TRUNC(SYSDATE) + 7` in the demo.

Q3. Why does book_ticket check sched_status?
Short answer: Bookings close once a journey is no longer SCHEDULED.
Detailed: If the status is DEPARTED, COMPLETED or CANCELLED, book_ticket raises ORA-20010 "Booking closed". Test suite 03 checks this.
Project connection: Q50.

Q10 · CREATE TABLE passenger

HOW TO EXPLAIN THIS TO THE EXAMINER

"PASSENGER stores travellers. Phone is mandatory, unique and validated by a regular expression for a 10-digit Indian mobile number starting with 6 to 9. Email is optional but unique when present — Oracle allows many NULLs in a unique column — and also regex-validated. registered_on defaults to SYSDATE."

Q1. Can a UNIQUE column contain several NULLs?
Short answer: Yes, in Oracle.
Detailed: NULL means unknown, so two NULLs are not considered equal, and Oracle does not store entirely-NULL keys in the unique index. Many passengers can therefore have no e-mail.
Project connection: uq_pass_email.

Q2. Why does the email CHECK pass when email is NULL?
Short answer: A CHECK fails only on FALSE.
Detailed: REGEXP_LIKE(NULL, pattern) returns UNKNOWN (NULL); constraints reject only FALSE, so UNKNOWN passes.
Project connection: ck_pass_email.

Q3. Why is phone VARCHAR2 and not NUMBER?
Short answer: It is an identifier, not a quantity.
Detailed: You never add phone numbers, leading zeros would be lost in other numbering formats, and the regex check needs text.
Project connection: ck_pass_phone.

Q4. What is the difference between DEFAULT SYSDATE and DEFAULT TRUNC(SYSDATE)?
Short answer: With or without the time.
Detailed: registered_on keeps the exact time; reservation.booking_date uses TRUNC so it can be compared with journey_date, which has no time.
Project connection: passenger.registered_on vs reservation.booking_date.

Q11 · CREATE TABLE reservation

HOW TO EXPLAIN THIS TO THE EXAMINER

"RESERVATION is the ticket table. Each row is one passenger on one journey in one class, between two stations, with a status. seat_id is a nullable foreign key because waitlisted and cancelled tickets hold no seat, and a CHECK constraint enforces that confirmed or completed tickets must have a seat while waitlisted or cancelled ones must not. The fare is left NULL in inserts and calculated by a BEFORE INSERT trigger."

Q1. Does Oracle check a foreign key when its value is NULL?
Short answer: No.
Detailed: An FK is checked only when all its columns are non-NULL. A NULL seat_id simply means "no seat"; to force a value you add NOT NULL.
Project connection: fk_res_seat.

Q2. Why can't "the seat must belong to this train" be a CHECK constraint?
Short answer: CHECK cannot look at other tables.
Detailed: A CHECK sees only the row being inserted. Verifying seat → coach → train needs queries on SEAT, COACH and TRAIN_SCHEDULE, so it is implemented in the trigger (rule R2).
Project connection: Q18.

Q3. Why is fare nullable when every ticket has a price?
Short answer: So the trigger can compute it.
Detailed: Callers omit the fare; the BEFORE trigger calculates it and assigns `:NEW.fare` before the row is written. The CHECK fare > 0 still applies to the final value.
Project connection: rule R4.

Q4. Explain ck_res_seat_status in plain words.
Short answer: Confirmed/completed ⇒ has a seat; waitlist/cancelled ⇒ no seat.
Detailed: It is an OR of two cases, each an AND of a status condition and a seat condition. Since every status is in exactly one of the two lists, each row must satisfy the case matching its status.
Project connection: test 5.5; data invariants 1 and 2 in test suite 02.

Q12 · Unique index ux_res_seat_journey

HOW TO EXPLAIN THIS TO THE EXAMINER

"This is a function-based unique index that prevents double-booking a seat on a journey. Its key is (schedule_id if a seat exists, seat_id). For waitlisted and cancelled tickets both parts are NULL, and Oracle doesn't store all-NULL keys in a B-tree index, so those rows never clash. A plain composite unique constraint would wrongly reject the second waitlisted ticket on the same journey."

Q1. What is a function-based index?
Short answer: An index on an expression rather than a plain column.
Detailed: Oracle evaluates the expression for each row and indexes the result (stored in a hidden virtual column). It can also speed up queries whose WHERE clause uses the same expression.
Project connection: `CASE WHEN seat_id IS NOT NULL THEN schedule_id END`.

Q2. Why does the index allow many waitlisted tickets on the same journey?
Short answer: Their key is entirely NULL, so it is not indexed.
Detailed: B-tree indexes skip rows whose key columns are all NULL. With no entry there is nothing to clash with.
Project connection: 20 WAITLIST tickets in the data.

Q3. Is a unique index the same as a unique constraint?
Short answer: Same effect, different object.
Detailed: A UNIQUE constraint is a declared rule that Oracle enforces with an index. A unique index enforces uniqueness directly. Only a constraint can be referenced by a foreign key, and only an index can be built on an expression like this one.
Project connection: uq_* constraints vs ux_* indexes.

Q4. How does this index protect against two simultaneous bookings?
Short answer: The second INSERT waits for, then fails on, the first.
Detailed: When session A inserts key (130, 3001) but has not committed, session B inserting the same key waits on A's row lock. If A commits, B gets ORA-00001 (DUP_VAL_ON_INDEX in PL/SQL); if A rolls back, B succeeds. book_ticket catches the error and tries the next seat.
Project connection: retry loop in Q50; tests/05 and test_concurrency_race.py.

Q13 · Unique index ux_res_pass_journey

HOW TO EXPLAIN THIS TO THE EXAMINER

"This index enforces that a passenger has at most one active ticket per journey. For cancelled tickets both key columns become NULL, so they are not indexed — which means a passenger who cancels can book the same journey again. Oracle doesn't support partial indexes with a WHERE clause, so this CASE trick gives the same result."

Q1. Why are both columns wrapped in CASE, not just one?
Short answer: To make the whole key NULL for cancelled rows.
Detailed: If only passenger_id were wrapped, a cancelled row would have key (NULL, schedule_id), which is partially non-NULL and therefore indexed; two cancelled tickets on the same journey would then clash.
Project connection: both CASE expressions.

Q2. Why have both a code check and an index for the same rule?
Short answer: Friendly message plus a guarantee.
Detailed: The procedure check raises ORA-20017 with a clear message. The index guarantees the rule for any INSERT path and under concurrency.
Project connection: book_ticket and test 03 step 4 (accepts -20017 or -1).

Q3. Can a passenger hold tickets on two different journeys?
Short answer: Yes.
Detailed: The key includes schedule_id, so (passenger 1, journey 101) and (passenger 1, journey 102) are different keys. Passenger 1's history in A1 shows several journeys.
Project connection: Q73 (A1).

Q14 · Supporting indexes

HOW TO EXPLAIN THIS TO THE EXAMINER

"Besides the two unique indexes, we create a composite index on (schedule_id, status), because almost every booking and reporting query asks for the tickets of one journey with a given status, plus indexes on foreign-key columns. Indexing foreign keys avoids Oracle taking a table-level lock on the child table when a parent row is deleted, and speeds up joins. The trade-off is slightly slower writes and extra storage."

Q1. Why does column order matter in a composite index?
Short answer: The index is sorted by the first column, then the second.
Detailed: (schedule_id, status) is like a phone book sorted by surname then first name: you can find all entries for a schedule quickly, but not all entries for a status without scanning.
Project connection: ix_res_schedule_status.

Q2. Does creating an index guarantee Oracle uses it?
Short answer: No.
Detailed: The cost-based optimizer chooses the cheapest plan using table statistics. For small tables or queries returning most rows, a full scan is often cheaper.
Project connection: 777-row RESERVATION table.

Q3. What is the downside of indexes?
Short answer: Slower writes and more storage.
Detailed: Every INSERT must add an entry to each index; UPDATEs of indexed columns must move entries; DELETEs must remove them.
Project connection: RESERVATION has 10 indexes.

Q15 · CREATE TABLE payment

HOW TO EXPLAIN THIS TO THE EXAMINER

"PAYMENT has a one-to-one relationship with RESERVATION: pnr is both a foreign key and unique, so each ticket has at most one payment. ON DELETE CASCADE removes the payment if the ticket is deleted. pay_status tracks refunds — SUCCESS, PARTIAL_REFUND or REFUNDED."

Q1. How do you make a relationship one-to-one in SQL?
Short answer: Put a UNIQUE constraint on the foreign key column.
Detailed: The FK alone allows many children per parent; UNIQUE limits it to one. NOT NULL additionally makes every payment belong to a ticket.
Project connection: uq_pay_pnr + fk_pay_res.

Q2. Why keep payment separate from reservation?
Short answer: Different entity, different lifecycle.
Detailed: It avoids nullable payment columns in RESERVATION and allows payment-specific rules and history.
Project connection: PAYMENT table.

Q3. What does ON DELETE CASCADE do here?
Short answer: Deleting a ticket deletes its payment.
Detailed: Without it, the DELETE on RESERVATION would fail with ORA-02292 while a payment exists.
Project connection: fk_pay_res.

Q16 · CREATE TABLE cancellation

HOW TO EXPLAIN THIS TO THE EXAMINER

"CANCELLATION is a 1-to-0..1 relationship with RESERVATION: pnr is unique and a foreign key, so a ticket is cancelled at most once. Keeping it separate avoids NULL columns on the many tickets that are never cancelled, and refund_amount defaults to zero with a non-negative CHECK."

Q1. What is the cardinality between RESERVATION and CANCELLATION?
Short answer: 1 to 0..1.
Detailed: Each cancellation belongs to exactly one ticket (NOT NULL FK); each ticket has zero or one cancellation (UNIQUE).
Project connection: uq_cancel_pnr.

Q2. How would you find tickets that were never cancelled?
Short answer: LEFT JOIN and test for NULL, or NOT EXISTS.
Detailed: `SELECT r.* FROM reservation r LEFT JOIN cancellation c ON c.pnr = r.pnr WHERE c.cancel_id IS NULL;`.
Project connection: Q5a uses LEFT JOIN to count cancellations.

Q3. Why can refund_amount be zero?
Short answer: Late cancellations get no refund.
Detailed: cancel_ticket refunds 0% if fewer than 4 hours remain before departure; the row still records that the ticket was cancelled.
Project connection: refund tiers in Q51.

Q17 · Sequences

HOW TO EXPLAIN THIS TO THE EXAMINER

"We use three sequences to generate PNRs, payment ids and cancellation ids. Each starts above the ids used in the sample data so there are no collisions. A sequence gives unique numbers even with concurrent users, but not gap-free ones, because a number used by a rolled-back transaction is never reused. NOCACHE keeps the numbering predictable."

Q1. Will PNRs always be consecutive?
Short answer: No, gaps are possible.
Detailed: NEXTVAL is not undone by ROLLBACK, and cached values can be lost on restart. Sequences guarantee uniqueness only.
Project connection: test suites roll back bookings.

Q2. What is the difference between NEXTVAL and CURRVAL?
Short answer: NEXTVAL advances; CURRVAL repeats the session's last value.
Detailed: CURRVAL returns the value most recently generated by NEXTVAL in your session; it is useful when inserting a parent and child with the same number.
Project connection: book_ticket uses `seq_pnr.NEXTVAL`.

Q3. Why does seq_pnr start at 4200001001?
Short answer: To stay clear of the sample PNRs.
Detailed: Sample tickets use 4200000001–4200000768. Starting above them avoids primary-key collisions with existing data.
Project connection: demo PNRs 4200001001–4200001009.

Q18 · Trigger trg_reservation_validate

HOW TO EXPLAIN THIS TO THE EXAMINER

"This is a BEFORE INSERT OR UPDATE, row-level trigger on RESERVATION. For every ticket it looks up the train and route of the journey, checks that both stations are on the route and that the boarding station comes first, checks that the seat belongs to a coach of this train and of the booked class, stops booking after the journey date, and if no fare is given it computes the fare as the greater of the distance and 50 km, times the class rate, plus the reservation fee. Because it's a BEFORE row trigger it can set :NEW.fare, and it never queries RESERVATION itself, so there's no mutating-table error."

Q1. What is the difference between a BEFORE and an AFTER trigger?
Short answer: BEFORE runs before the row is written and can change :NEW; AFTER runs after and cannot.
Detailed: BEFORE is used for validation and derived values; AFTER is used for auditing or actions that need the final row (e.g. writing to a log table).
Project connection: fare is assigned to `:NEW.fare`.

Q2. How many times does the trigger fire for `INSERT INTO reservation SELECT …` returning 768 rows?
Short answer: 768 times.
Detailed: FOR EACH ROW triggers fire once per affected row. A statement-level trigger would fire once.
Project connection: 02_data.sql loads 768 tickets.

Q3. What is a mutating table error and why doesn't it happen here?
Short answer: ORA-04091: a row trigger read or changed its own table; this trigger never touches RESERVATION.
Detailed: While a statement is changing rows, the table is in an inconsistent intermediate state, so Oracle forbids row triggers from querying it. Our trigger only reads other tables.
Project connection: all SELECTs in Q18 are on other tables.

Q4. Why is the fare calculated with GREATEST(distance, 50)?
Short answer: A minimum chargeable distance of 50 km.
Detailed: Very short trips are charged as 50 km, like Indian Railways' minimum fare rule. For KPD→JTJ (84 km) the real distance is used; a 30 km trip would be charged for 50.
Project connection: rule R4; test suite 02 invariant 8 re-checks every fare with the same formula.

Q5. Does the trigger fire for `UPDATE reservation SET status = 'COMPLETED'`?
Short answer: No.
Detailed: The UPDATE OF list contains six columns, not status. The trigger fires only if at least one listed column appears in the SET clause.
Project connection: but cancel_ticket's `SET status='CANCELLED', seat_id=NULL` does fire it (seat_id is listed); with seat NULL, R2 is skipped and fare is kept.

Q6. Why use RAISE_APPLICATION_ERROR instead of RAISE?
Short answer: To give a custom error code and message.
Detailed: RAISE_APPLICATION_ERROR(-20000…-20999, 'text') returns a meaningful message to the caller and rolls back the triggering statement. The caller can catch it by code (SQLCODE = -20002).
Project connection: codes -20001 to -20004.

4. Files 02_data.sql and 02b_seat.sql — Loading the Sample Data

Q19 · INSERT stations (2.1)

HOW TO EXPLAIN THIS TO THE EXAMINER

"These are simple positional INSERTs that load the 14 stations. They run first because every route, stop and ticket has a foreign key to STATION. Each insert is checked against the primary key and the CHECK constraints, such as the uppercase station code."

Q1. What happens if one station INSERT fails?
Short answer: Only that statement is undone.
Detailed: Oracle uses statement-level atomicity: the failing INSERT is rolled back, the previous successful INSERTs remain pending in the transaction, and the script continues (WHENEVER SQLERROR CONTINUE).
Project connection: run_all.sql setting.

Q2. When does this data become permanent?
Short answer: At the next COMMIT, or at the next DDL statement.
Detailed: 02_data.sql ends with COMMIT. Any DDL statement would also commit implicitly.
Project connection: last line of 02_data.sql.

Q3. Why must stations be inserted before routes?
Short answer: Foreign keys.
Detailed: route.origin_code and dest_code reference station; the parent row must exist or the child INSERT fails with ORA-02291.
Project connection: fk_route_origin/fk_route_dest.

Q20 · INSERT routes and route stops (2.2)

HOW TO EXPLAIN THIS TO THE EXAMINER

"We insert the four routes, then their stops in order with the distance of each stop from the origin. ROUTE_STOP is checked against two foreign keys and two unique keys, so a station can't appear twice on a route and a position can't be used twice. These distances drive fare calculation."

Q1. How is the distance from Katpadi to Bengaluru found?
Short answer: 359 − 129 = 230 km.
Detailed: Both stops are on route 1; the trigger subtracts their km_from_origin values.
Project connection: PNR 4200000002 fare 374.

Q2. Why store km per stop instead of per station pair?
Short answer: Fewer rows, no inconsistency.
Detailed: n stops need n rows; storing every pair needs n(n−1)/2 rows that could disagree with each other.
Project connection: ROUTE_STOP design.

Q3. Is route_name used as a key anywhere?
Short answer: Reports group by it.
Detailed: Q3, Q6a, Q9, Q10 and A4 group by route_name; UNIQUE guarantees each name is one route.
Project connection: uq_route_name.

Q21 · INSERT coach classes (2.3)

HOW TO EXPLAIN THIS TO THE EXAMINER

"This loads the six classes. Each row holds the fare rule and the seats per coach, which the trigger uses for fares and the views use for capacity. Seats per coach are deliberately small so trains fill and waitlists appear in the demo."

Q1. What is the fare for 100 km in EC?
Short answer: 100 × 2.90 + 60 = ₹350.
Detailed: The trigger uses GREATEST(100, 50) = 100 km, times 2.90, plus the 60 fee.
Project connection: rule R4.

Q2. How is a train's capacity calculated?
Short answer: Sum of seats_per_coach over its coaches.
Detailed: `v_reservation_performance` joins COACH to COACH_CLASS and sums seats_per_coach per train.
Project connection: Q56.

Q3. Why must classes exist before coaches?
Short answer: coach.class_code is a foreign key.
Detailed: fk_coach_class requires the class row to exist.
Project connection: 2.3 runs before 2.5.

Q22 · INSERT trains (2.4)

HOW TO EXPLAIN THIS TO THE EXAMINER

"These INSERTs name their columns, so run_days and is_active are filled by their DEFAULTs. Each train is checked for a valid 5-digit number, an allowed type, a valid HH:MM time by regex, and an existing route."

Q1. What values do run_days and is_active get?
Short answer: 'YYYYYYY' and 'Y'.
Detailed: They are not in the column list, so their DEFAULT expressions are used.
Project connection: T_TRAIN output.

Q2. Why prefer an explicit column list?
Short answer: Safer and self-documenting.
Detailed: It does not depend on column order and still works after new nullable or defaulted columns are added.
Project connection: compare with positional INSERTs for STATION.

Q3. Which trains share route 1?
Short answer: 12639, 12607 and 12027.
Detailed: Three trains run Chennai–Bengaluru; Q3 aggregates them per route and A2 compares each with its route average.
Project connection: Q62, Q74.

Q23 · INSERT coaches (2.5)

HOW TO EXPLAIN THIS TO THE EXAMINER

"These rows give each train its coaches. The coach's class decides which seats exist and the train's capacity. Coach ids are chosen so seat ids follow the pattern coach_id times 100 plus seat number, which makes test data easy to read."

Q1. What is the capacity of train 12639 per journey?
Short answer: 32 seats.
Detailed: Two CC coaches × 12 + one EC coach × 8 = 32.
Project connection: capacity column in v_reservation_performance.

Q2. Which train has a First AC coach?
Short answer: 12671 (coach H1).
Detailed: Coach 27 is H1, class 1A, with 4 seats.
Project connection: T_COACH output.

Q3. What happens if you book class 2A on train 12639?
Short answer: ORA-20011 "Class 2A is not available on this train".
Detailed: book_ticket counts coaches of that class on the journey's train; zero means unavailable.
Project connection: Q50.

Q24 · Seat generator: INSERT … SELECT with CONNECT BY (2.6 and 02b_seat.sql)

HOW TO EXPLAIN THIS TO THE EXAMINER

"Instead of 320 separate inserts, we generate all seats with one INSERT…SELECT. A CONNECT BY LEVEL query on DUAL produces the numbers 1 to 100, and we join each coach to those numbers up to its class's seats_per_coach. The seat id is coach_id times 100 plus the seat number, and MOD with DECODE assigns berth types in repeating bays. The 02b version adds NOT EXISTS so it only inserts missing seats and can be run any number of times."

Q1. What does `SELECT LEVEL FROM dual CONNECT BY LEVEL <= 100` return?
Short answer: 100 rows numbered 1–100.
Detailed: CONNECT BY builds a hierarchy; DUAL has one row, and the condition lets it connect to itself until LEVEL exceeds 100, so each level becomes one row.
Project connection: row generator `n`.

Q2. What berth is seat 8 of a sleeper coach?
Short answer: SIDE UPPER.
Detailed: MOD(8, 8) = 0, which matches none of 1–7, so DECODE returns its default 'SIDE UPPER'.
Project connection: SL/3A DECODE.

Q3. Why does 02b_seat.sql use NOT EXISTS?
Short answer: To make it idempotent.
Detailed: It inserts only seats that are missing, so it can be run after 02_data.sql (adds 0 rows) or alone after coaches are loaded.
Project connection: 02b_seat.sql.

Q4. Is `JOIN … ON n.lvl <= cc.seats_per_coach` an equi-join?
Short answer: No, a non-equi join.
Detailed: It matches rows by an inequality. Any comparison operator can be used in ON.
Project connection: seat generator.

Q5. What is the difference between CASE and DECODE?
Short answer: CASE is ANSI standard and more flexible; DECODE is Oracle-only equality matching.
Detailed: CASE supports any condition (ranges, IN, AND); DECODE only compares one expression with values. DECODE treats NULL = NULL as a match.
Project connection: both are used in Q24.

Q25 · INSERT train schedules (2.7)

HOW TO EXPLAIN THIS TO THE EXAMINER

"We create three journeys per train — two completed, one scheduled — using ANSI date literals, which have no time part and therefore pass the CHECK that journey dates are truncated. The unique constraint on train and date stops duplicate runs."

Q1. Why use `DATE '2026-09-20'` and not `'20-SEP-2026'`?
Short answer: It doesn't depend on session settings.
Detailed: The ANSI literal is always YYYY-MM-DD; a string relies on implicit conversion with NLS settings.
Project connection: all date inserts.

Q2. How many journeys does each train have?
Short answer: Three (and 12677 has a fourth, demo journey 130).
Detailed: Ids 101–127 cover 9 trains × 3; the demo adds 130 for 12677.
Project connection: v_train_utilization `journeys` column.

Q3. Can you still book journey 103 today?
Short answer: No.
Detailed: Its date (21-Oct-2026) is before today's booking date; trigger rule R3 raises ORA-20004.
Project connection: Q18 R3.

Q26 · INSERT passengers (2.8)

HOW TO EXPLAIN THIS TO THE EXAMINER

"This loads 150 passengers with positional inserts. Each phone is validated by a regular expression and must be unique, and so must the e-mail. The data covers different ages and cities so that the concession update and the city-based queries have something to work with."

Q1. Would these INSERTs still work after Part 3 of 03_dml.sql?
Short answer: No.
Detailed: Part 3 adds a ninth column (concession_type) and renames city. Positional 8-value INSERTs would give ORA-00947 not enough values.
Project connection: Q32, Q34, Q42.

Q2. Why is registered_on given explicitly?
Short answer: To simulate past registrations.
Detailed: The DEFAULT would set everyone to today; explicit dates give realistic history.
Project connection: passenger 1 registered 28-Jan-2025.

Q3. Which constraint stops two passengers sharing a phone?
Short answer: uq_pass_phone.
Detailed: A UNIQUE constraint backed by a unique index; a duplicate gives ORA-00001.
Project connection: PASSENGER DDL.

Q27 · INSERT reservations (2.9)

HOW TO EXPLAIN THIS TO THE EXAMINER

"We load 768 tickets without a fare column. For each one the BEFORE INSERT trigger validates the route direction and seat, and computes the fare from the distance and the class rate. Then the constraints check that confirmed tickets have seats, waitlisted and cancelled ones don't, and the unique indexes stop any seat being sold twice on a journey."

Q1. Where does the fare of PNR 4200000001 come from?
Short answer: The trigger: 359 × 1.45 + 40 = 560.55 → 561.
Detailed: The INSERT omits fare, so `:NEW.fare` is NULL and rule R4 computes it.
Project connection: T_TRIGFARE output.

Q2. How many times does the trigger fire during this section?
Short answer: 768 times.
Detailed: It is a row-level trigger and each INSERT inserts one row.
Project connection: Q18.

Q3. Why do cancelled tickets in the data have seat_id NULL?
Short answer: ck_res_seat_status requires it.
Detailed: A cancelled ticket must not hold a seat; otherwise the seat could never be resold on that journey.
Project connection: PNR 4200000006.

5. File 03_dml.sql — Modification, Integrity Tests and Stored Procedures

Q28 · Generate payments with INSERT … SELECT (2.10)

HOW TO EXPLAIN THIS TO THE EXAMINER

"Payments are generated from the reservations with one INSERT…SELECT. ROW_NUMBER over PNR order gives sequential payment ids, the amount is the ticket's fare, DECODE on MOD spreads the payment modes, and a CASE sets the status — partial refund for cancelled tickets, full refund for waitlisted tickets on journeys that have already run, and success otherwise."

Q1. What does ROW_NUMBER() OVER (ORDER BY r.pnr) do?
Short answer: Numbers the result rows 1, 2, 3… in PNR order.
Detailed: It is an analytic function: it computes a value for each row using a window of rows, without collapsing them as GROUP BY would.
Project connection: payment_id = 900000 + row number.

Q2. Why are some payments REFUNDED?
Short answer: Waitlisted tickets on completed journeys never got a seat.
Detailed: Their money is returned in full; the CASE identifies them by status WAITLIST and sched_status COMPLETED.
Project connection: 15 REFUNDED payments.

Q3. Is this a single transaction step?
Short answer: It is one statement, so it is atomic.
Detailed: Either all 768 rows are inserted or none (if any row fails, the whole statement is rolled back).
Project connection: statement-level atomicity.

Q29 · Generate cancellations with INSERT … SELECT (2.11)

HOW TO EXPLAIN THIS TO THE EXAMINER

"Cancellation records are generated from the cancelled tickets with INSERT…SELECT. Each cancellation date is placed halfway between booking and journey using date arithmetic, and the refund is 90, 75 or 50 percent depending on how many days were left. For PNR 4200000006 that gives 504.90 out of 561."

Q1. What is `date1 - date2` in Oracle?
Short answer: The number of days between them.
Detailed: It can be fractional if times are involved; adding a number to a date adds days.
Project connection: midpoint calculation.

Q2. Why FLOOR?
Short answer: To get a whole number of days.
Detailed: Halving an odd number of days gives .5; FLOOR keeps the cancellation date at midnight-aligned whole days.
Project connection: `FLOOR((journey - booking) / 2)`.

Q3. When does this data become permanent?
Short answer: At the COMMIT that follows.
Detailed: COMMIT ends the transaction started by the first INSERT in 02_data.sql.
Project connection: `COMMIT;` after 2.11.

Q30 · Transaction control in the scripts (COMMIT and ROLLBACK)

HOW TO EXPLAIN THIS TO THE EXAMINER

"Oracle starts a transaction automatically with the first DML and ends it with COMMIT or ROLLBACK, and every DDL statement commits implicitly. The script commits at the end of each logical part, and the what-if analysis ends with ROLLBACK so the threshold change is undone — that demonstrates atomicity and isolation."

Q1. What are the ACID properties?
Short answer: Atomicity, Consistency, Isolation, Durability.
Detailed: All-or-nothing; constraints always hold; concurrent transactions don't see each other's uncommitted work; committed changes are permanent.
Project connection: book_ticket inserts a reservation and a payment in one transaction.

Q2. Does Oracle need BEGIN TRANSACTION?
Short answer: No.
Detailed: A transaction begins implicitly with the first DML statement after a COMMIT/ROLLBACK.
Project connection: none of the scripts use BEGIN TRANSACTION.

Q3. What is a SAVEPOINT and does the project use one?
Short answer: A marker for partial rollback; not used.
Detailed: `SAVEPOINT a; … ROLLBACK TO a;` undoes work after the savepoint only. The project uses only full COMMIT/ROLLBACK.
Project connection: supplementary.

Q31 · Row counts with UNION ALL

HOW TO EXPLAIN THIS TO THE EXAMINER

"This sanity check counts the rows of every table and stacks the twelve single-row results with UNION ALL. We use UNION ALL rather than UNION because there can't be duplicates and UNION ALL avoids the extra sort."

Q1. UNION vs UNION ALL?
Short answer: UNION removes duplicates; UNION ALL keeps all rows.
Detailed: UNION performs a sort or hash to eliminate duplicates; UNION ALL simply appends results and is faster.
Project connection: also used in A5 to combine boarding and alighting stations.

Q2. Where do the column names of the result come from?
Short answer: The first SELECT.
Detailed: Aliases in later SELECTs are ignored.
Project connection: `tbl`, `row_count`.

Q3. Why is no GROUP BY needed?
Short answer: Each SELECT aggregates its whole table into one group.
Detailed: An aggregate with no GROUP BY treats all rows as one group and returns one row (even for an empty table, COUNT(*) = 0).
Project connection: each branch.

Q32 · ALTER TABLE passenger ADD concession_type (3.1)

HOW TO EXPLAIN THIS TO THE EXAMINER

"This ALTER adds a concession_type column to a table that already has data. Because it's NOT NULL, it needs a DEFAULT — Oracle fills all existing rows with 'NONE'. A CHECK limits it to four categories."

Q1. Can you add a NOT NULL column to a table with rows?
Short answer: Only with a DEFAULT.
Detailed: Otherwise existing rows would violate NOT NULL; Oracle raises ORA-01758.
Project connection: concession_type.

Q2. Can this ALTER be rolled back?
Short answer: No, DDL commits implicitly.
Detailed: You would need another ALTER (DROP COLUMN) to undo it.
Project connection: Part 3.

Q3. Which statements use the new column?
Short answer: The concession UPDATEs in Part 4.1.
Detailed: They set SENIOR or STUDENT from the date of birth.
Project connection: Q36.

Q33 · ALTER TABLE station MODIFY (3.2)

HOW TO EXPLAIN THIS TO THE EXAMINER

"MODIFY widens station_name to 80 characters. Widening is always allowed; narrowing or changing the type would fail if existing data doesn't fit."

Q1. Can a column be shortened?
Short answer: Only if all existing values fit.
Detailed: Otherwise ORA-01441.
Project connection: station_name.

Q2. Does MODIFY remove the NOT NULL constraint?
Short answer: No.
Detailed: Only the attributes written in MODIFY change.
Project connection: station_name stays NOT NULL.

Q3. ADD vs MODIFY?
Short answer: ADD creates a new column; MODIFY changes an existing one.
Detailed: Both are ALTER TABLE clauses; RENAME COLUMN and DROP COLUMN are others.
Project connection: Q32 vs Q33.

Q34 · ALTER TABLE passenger RENAME COLUMN (3.3)

HOW TO EXPLAIN THIS TO THE EXAMINER

"RENAME COLUMN changes city to home_city to avoid confusion with STATION's city. It's done before any query uses the column, which is why later code refers to home_city."

Q1. Does renaming a column lose data?
Short answer: No.
Detailed: Only the dictionary name changes.
Project connection: home_city keeps all 150 values.

Q2. Why does the execution order matter here?
Short answer: Later code uses the new name.
Detailed: Q42 and 04_queries.sql would fail with ORA-00904 if run before the rename.
Project connection: dependency in Chapter 1.

Q3. Why rename at all?
Short answer: Clarity.
Detailed: `city` exists in STATION too; `home_city` makes queries joining both unambiguous to readers.
Project connection: Q8 selects p.home_city alongside station data via views.

Q35 · ALTER TABLE train ADD CONSTRAINT (3.4)

HOW TO EXPLAIN THIS TO THE EXAMINER

"This adds a unique constraint to an existing table. Oracle validates the current rows first and builds a unique index; since all train names are distinct it succeeds, and from then on a duplicate name is rejected."

Q1. What happens if existing data violates a new constraint?
Short answer: The ALTER fails.
Detailed: ORA-02299 for UNIQUE, ORA-02293 for CHECK; nothing changes.
Project connection: uq_train_name.

Q2. Does a UNIQUE constraint create an index?
Short answer: Yes, a unique index (unless a suitable one exists).
Detailed: The index enforces uniqueness efficiently.
Project connection: UQ_TRAIN_NAME in the index list.

Q3. Why does this matter for reports?
Short answer: Grouping by name is safe.
Detailed: Without uniqueness, two trains with the same name would be merged in Q9's ROLLUP by train_name.
Project connection: Q71 (report Q9).

Q36 · UPDATE concession from age (4.1)

HOW TO EXPLAIN THIS TO THE EXAMINER

"These UPDATEs compute each passenger's age with MONTHS_BETWEEN(SYSDATE, dob) divided by 12 and set SENIOR for 60 and above, STUDENT for under 23. The SELECT verifies the result. Because it uses SYSDATE, the result depends on the day the script runs — today it gave 9 seniors and 17 students."

Q1. What happens if you forget the WHERE in an UPDATE?
Short answer: All rows are updated.
Detailed: Always run a SELECT with the same WHERE first, and don't COMMIT until checked.
Project connection: concession UPDATE.

Q2. Why MONTHS_BETWEEN instead of subtracting dates?
Short answer: Leap-year accuracy.
Detailed: Date subtraction gives days; dividing by 365 is slightly wrong over decades. MONTHS_BETWEEN counts calendar months.
Project connection: age calculation.

Q3. Why might the output differ next year?
Short answer: SYSDATE changes.
Detailed: Ages increase, so more passengers turn 60 and some stop being under 23.
Project connection: session-dependent result.

Q37 · UPDATE train timetable (4.2)

HOW TO EXPLAIN THIS TO THE EXAMINER

"A simple UPDATE by primary key changes the departure time of 12639. The CHECK constraint validates the new time, and because we filter by the key exactly one row changes."

Q1. Is updating 0 rows an error?
Short answer: No.
Detailed: SQL reports "0 rows updated"; in PL/SQL check SQL%ROWCOUNT.
Project connection: cancel_ticket checks SQL%ROWCOUNT.

Q2. Does the reservation trigger fire?
Short answer: No.
Detailed: The trigger is on RESERVATION, not TRAIN.
Project connection: Q18.

Q3. Which procedure uses dep_time?
Short answer: cancel_ticket.
Detailed: It converts HH:MM into a fraction of a day to compute the exact departure moment.
Project connection: Q51.

Q38 · Find a PNR into a bind variable (4.3)

HOW TO EXPLAIN THIS TO THE EXAMINER

"We declare a SQL*Plus bind variable and use a PL/SQL block to store the smallest PNR of a confirmed MAS-to-SBC ticket on journey 103. Using MIN guarantees exactly one row, so SELECT INTO can't fail with too many rows."

Q1. What is a bind variable?
Short answer: A placeholder whose value is supplied separately from the SQL text.
Detailed: It lets the same statement run with different values, avoids re-parsing, and prevents SQL injection in applications.
Project connection: :v_pnr.

Q2. Why can't SELECT INTO here raise NO_DATA_FOUND?
Short answer: An aggregate without GROUP BY always returns one row.
Detailed: If no rows match, MIN returns NULL in that single row.
Project connection: MIN(pnr).

Q3. Where is :v_pnr used next?
Short answer: In the before/after SELECTs and both UPDATEs.
Detailed: Q39–Q41.
Project connection: Part 4.3.

Q39 · Before-and-after check of the ticket and payment (4.3)

HOW TO EXPLAIN THIS TO THE EXAMINER

"This joins the ticket with its payment and is run before and after the modification. Before, the ticket is MAS to SBC with fare and payment 561; after, it is KPD to SBC with 374 for both."

Q1. Why is this an inner join?
Short answer: Every ticket has exactly one payment.
Detailed: The 1:1 relationship means no rows are lost.
Project connection: uq_pay_pnr.

Q2. What does `AS paid` do?
Short answer: Renames the output column.
Detailed: Column aliases affect only the result heading; they can be used in ORDER BY but not in WHERE.
Project connection: `p.amount AS paid`.

Q3. Why did the fare change from 561 to 374?
Short answer: Shorter distance: 230 km instead of 359.
Detailed: 230 × 1.45 + 40 = 373.5 → 374.
Project connection: Q40.

Q40 · UPDATE boarding station and recalculate fare (4.3)

HOW TO EXPLAIN THIS TO THE EXAMINER

"To change the boarding point we update from_station and set fare to NULL in the same statement. The BEFORE UPDATE trigger re-validates the route and, seeing a NULL fare, recalculates it for the shorter distance — 374 instead of 561."

Q1. Why set fare to NULL?
Short answer: To trigger recalculation.
Detailed: Rule R4 only computes a fare when :NEW.fare IS NULL.
Project connection: trigger R4.

Q2. Does the trigger fire on this UPDATE?
Short answer: Yes.
Detailed: from_station and fare are both in the UPDATE OF list.
Project connection: Q18 header.

Q3. Which order do the trigger and the CHECK constraints run in?
Short answer: BEFORE row trigger first, then constraints.
Detailed: Oracle fires BEFORE triggers, then validates constraints on the final row values.
Project connection: fare is non-NULL by the time ck_res_fare is checked.

Q41 · UPDATE payment using a scalar subquery (4.3)

HOW TO EXPLAIN THIS TO THE EXAMINER

"This UPDATE uses a scalar subquery to copy the recalculated fare into the payment, so payment amount and fare stay equal, which is one of our data invariants."

Q1. What is a scalar subquery?
Short answer: A subquery returning one value.
Detailed: One row, one column; more rows raise ORA-01427; no rows gives NULL.
Project connection: SET amount = (SELECT fare …).

Q2. Could this be done in the trigger?
Short answer: Possibly with an AFTER trigger, but not in the existing one.
Detailed: A BEFORE row trigger on RESERVATION could update PAYMENT, but mixing responsibilities makes triggers harder to reason about; explicit code is clearer.
Project connection: design choice.

Q3. What if the outer WHERE is missing?
Short answer: All payments get the same amount.
Detailed: The subquery still returns 374, applied to every row.
Project connection: importance of WHERE.

Q42 · INSERT then DELETE a test passenger (4.4)

HOW TO EXPLAIN THIS TO THE EXAMINER

"We insert a test passenger with a column list — needed since the table now has nine columns — and delete it. It can be deleted because no ticket references it; a passenger with tickets is protected by the foreign key, as test 5.6 shows."

Q1. DELETE vs TRUNCATE vs DROP?
Short answer: Rows / all rows fast / whole table.
Detailed: DELETE is DML, can have WHERE, fires triggers, can roll back. TRUNCATE is DDL, removes all rows, cannot roll back, resets storage. DROP removes the table and its definition.
Project connection: DELETE here; DROP in Q01.

Q2. Why does this DELETE succeed but test 5.6 fail?
Short answer: Passenger 999 has no tickets.
Detailed: fk_res_pass has no ON DELETE clause, so deletion is restricted only when children exist.
Project connection: Q48.

Q3. What values do registered_on and concession_type get?
Short answer: SYSDATE and 'NONE'.
Detailed: They are not in the column list, so their DEFAULTs apply.
Project connection: DEFAULTs from Q10 and Q32.

Q43 · Test 5.1 — invalid phone

HOW TO EXPLAIN THIS TO THE EXAMINER

"Test 5.1 inserts a passenger with phone 12345. The regex CHECK constraint rejects it with ORA-02290, which the exception handler catches and prints, proving domain integrity is enforced by the database."

Q1. What is SQLERRM?
Short answer: The error message of the current exception.
Detailed: SQLCODE gives the number (e.g. -2290).
Project connection: all Part 5 handlers.

Q2. Is anything inserted?
Short answer: No.
Detailed: The failing statement is rolled back.
Project connection: statement-level atomicity.

Q3. Why doesn't the script stop?
Short answer: The exception is handled inside the block.
Detailed: A handled exception ends the block normally.
Project connection: EXCEPTION WHEN OTHERS.

Q44 · Test 5.2 — double seat booking

HOW TO EXPLAIN THIS TO THE EXAMINER

"Test 5.2 tries to sell an already-sold seat on journey 101. The trigger accepts it, but the function-based unique index ux_res_seat_journey rejects it with ORA-00001, so double booking is impossible at database level."

Q1. Which object rejects this?
Short answer: The unique index ux_res_seat_journey.
Detailed: Key (schedule 101, seat 101) already exists.
Project connection: Q12.

Q2. What is SYS_NC00011$?
Short answer: A hidden virtual column for the CASE expression.
Detailed: Oracle creates it to store the function-based index key.
Project connection: error text.

Q3. In PL/SQL, which named exception is ORA-00001?
Short answer: DUP_VAL_ON_INDEX.
Detailed: book_ticket catches it to retry with the next seat.
Project connection: Q50.

Q45 · Test 5.3 — seat of another train

HOW TO EXPLAIN THIS TO THE EXAMINER

"Test 5.3 gives a journey on train 12639 a seat that belongs to train 12677. The foreign key is satisfied because the seat exists, but the trigger's rule R2 checks the seat's coach and train and raises ORA-20003."

Q1. Why can't a foreign key prevent this?
Short answer: It only checks existence.
Detailed: Ownership needs a join across SEAT, COACH and TRAIN_SCHEDULE.
Project connection: rule R2.

Q2. What does ORA-04088 mean?
Short answer: The error happened inside a trigger.
Detailed: It wraps the original error raised in the trigger.
Project connection: error stack.

Q3. How is seat 2801 decoded?
Short answer: Coach 28, seat 1.
Detailed: seat_id = coach_id × 100 + seat_no.
Project connection: Q24.

Q46 · Test 5.4 — wrong direction

HOW TO EXPLAIN THIS TO THE EXAMINER

"Test 5.4 tries to travel SBC to MAS on a train that runs MAS to SBC. The trigger compares the stop sequence numbers, 4 and 1, and raises ORA-20002."

Q1. How does the trigger know the direction?
Short answer: It compares stop_seq values.
Detailed: Boarding stop_seq must be smaller than destination stop_seq.
Project connection: R1.

Q2. Why use WAITLIST in this test?
Short answer: To avoid seat rules.
Detailed: No seat means R2 and the seat index are not involved.
Project connection: test design.

Q3. Would `from_station = to_station` hit the same error?
Short answer: Yes, in the trigger (equal seq) — before the CHECK.
Detailed: BEFORE triggers run before constraints are checked.
Project connection: ck_res_stations would otherwise catch it.

Q47 · Test 5.5 — confirmed ticket without a seat

HOW TO EXPLAIN THIS TO THE EXAMINER

"Test 5.5 inserts a confirmed ticket without a seat. A fixed booking date keeps the trigger happy, so the CHECK constraint ck_res_seat_status is what rejects it with ORA-02290."

Q1. Why is booking_date given explicitly?
Short answer: So R3 doesn't fail first.
Detailed: The default (today) is after journey 103's date.
Project connection: trigger R3.

Q2. Which runs first, trigger or CHECK?
Short answer: The BEFORE trigger.
Detailed: Constraint checks happen on the final row after BEFORE triggers.
Project connection: error order.

Q3. How would a WAITLIST ticket with a seat fail?
Short answer: Same CHECK, ORA-02290.
Detailed: The second branch requires seat_id IS NULL for WAITLIST.
Project connection: ck_res_seat_status.

Q48 · Test 5.6 — delete a parent row

HOW TO EXPLAIN THIS TO THE EXAMINER

"Test 5.6 tries to delete passenger 1, who has tickets. The foreign key from RESERVATION has no cascade, so Oracle refuses with ORA-02292, child record found. Ticket history can't be orphaned."

Q1. ORA-02291 vs ORA-02292?
Short answer: Parent not found vs child found.
Detailed: 02291 on inserting/updating a child with no parent; 02292 on deleting/updating a parent that has children.
Project connection: tests 5.7 and 5.6.

Q2. What delete options exist for FKs in Oracle?
Short answer: Default restrict, ON DELETE CASCADE, ON DELETE SET NULL.
Detailed: Oracle has no ON UPDATE CASCADE.
Project connection: cascades used for payment/cancellation.

Q3. Why does the index on passenger_id matter here?
Short answer: Fast child lookup, no table lock.
Detailed: Without it, Oracle scans and locks RESERVATION during the check.
Project connection: ix_res_passenger.

Q49 · Test 5.7 — unknown station

HOW TO EXPLAIN THIS TO THE EXAMINER

"Test 5.7 inserts a route stop for a station code that doesn't exist. The foreign key to STATION rejects it with ORA-02291, parent key not found. Together the seven tests show every kind of integrity rule being enforced."

Q1. Which constraint fires?
Short answer: fk_rs_station.
Detailed: 'XYZ' has no parent in STATION.
Project connection: ROUTE_STOP DDL.

Q2. List the integrity types demonstrated in Part 5.
Short answer: Domain, key/unique, referential, and trigger-based business rules.
Detailed: 5.1 CHECK; 5.2 unique index; 5.3/5.4 trigger; 5.5 cross-column CHECK; 5.6/5.7 FK.
Project connection: Part 5.

Q3. What is a deferrable constraint?
Short answer: One checked at COMMIT instead of per statement.
Detailed: Useful for circular references; not used in this project.
Project connection: supplementary.

Q50 · Procedure book_ticket (6.1)

HOW TO EXPLAIN THIS TO THE EXAMINER

"book_ticket is the booking counter. It checks that the passenger and journey exist, locks the journey row with SELECT FOR UPDATE so concurrent bookings for the same journey are serialised, checks the journey is open, that the passenger has no other active ticket and that the class exists on the train. Then it finds the lowest-numbered free seat of that class using NOT EXISTS against the journey's reservations. If there is none the ticket is waitlisted. It inserts the reservation — the trigger computes the fare, which RETURNING INTO gives back — and inserts the payment. If the unique index reports the seat was taken concurrently, it retries with the next seat, up to 50 times. It doesn't commit; the caller does."

Q1. What is the difference between IN and OUT parameters?
Short answer: IN passes a value in; OUT passes a value back.
Detailed: IN parameters are read-only inside the procedure. OUT parameters start as NULL and are assigned inside; the caller must pass variables for them. IN OUT does both.
Project connection: p_pnr and p_status are OUT.

Q2. What does SELECT … FOR UPDATE do?
Short answer: Locks the selected rows until COMMIT or ROLLBACK.
Detailed: Other sessions can still read the row (read consistency), but cannot lock or update it; they wait. This serialises bookings per journey.
Project connection: train_schedule row of p_schedule. Test suite 05 checks the lock with FOR UPDATE NOWAIT.

Q3. How is a waitlist decided?
Short answer: When the free-seat query finds no row.
Detailed: NO_DATA_FOUND is caught, v_seat stays NULL, status becomes WAITLIST, and the CHECK constraint accepts it because waitlisted tickets must have no seat.
Project connection: passenger 19 in the demo.

Q4. Why is there no COMMIT in the procedure?
Short answer: Transaction control belongs to the caller.
Detailed: The caller may need the booking to succeed or fail together with other work, and test scripts roll back their bookings.
Project connection: demo commits at the end of Part 6; tests ROLLBACK.

Q5. What is RETURNING INTO?
Short answer: It returns column values of the row just inserted or updated.
Detailed: Here it fetches the fare computed by the BEFORE trigger without a second SELECT.
Project connection: `RETURNING fare INTO v_fare`.

Q6. Why both a procedure check and a unique index for "one ticket per passenger per journey"?
Short answer: Clear error message plus guaranteed enforcement.
Detailed: The check raises -20017 with a message; the index catches any path that bypasses the procedure.
Project connection: -20017 and ux_res_pass_journey.

Q7. What happens if DUP_VAL_ON_INDEX occurs on a waitlist insert?
Short answer: It is re-raised.
Detailed: A waitlist row has no seat, so the duplicate must come from ux_res_pass_journey; retrying would not help.
Project connection: `ELSE RAISE;`.

Q51 · Procedure cancel_ticket (6.2)

HOW TO EXPLAIN THIS TO THE EXAMINER

"cancel_ticket locks the ticket row, computes the exact departure time from the journey date and the train's HH:MM time, and refuses if the ticket is already cancelled or completed or the train has left. It refunds 100 percent for waitlisted tickets and 90, 75, 50 or 0 percent depending on hours left. It sets the ticket to CANCELLED and clears the seat, records the cancellation, updates the payment status, and if a seat was freed, promotes the earliest waitlisted ticket of the same journey and class to CONFIRMED on that seat, re-checking its status to be safe under concurrency."

Q1. How is the departure time calculated?
Short answer: journey_date + hours/24 + minutes/1440.
Detailed: Oracle dates add fractions of a day; SUBSTR and TO_NUMBER extract hours and minutes from 'HH:MM'.
Project connection: v_depart.

Q2. What does `FOR UPDATE OF r.status` lock?
Short answer: Only the reservation row.
Detailed: In a join, FOR UPDATE without OF locks rows of all joined tables; OF restricts it to the named table's rows.
Project connection: the train and schedule rows stay unlocked.

Q3. What is SQL%ROWCOUNT?
Short answer: Rows affected by the last SQL statement.
Detailed: An implicit cursor attribute; others are SQL%FOUND, SQL%NOTFOUND, SQL%ISOPEN.
Project connection: promotion loop.

Q4. Why must status and seat_id be updated in the same statement?
Short answer: Constraints are checked per statement.
Detailed: Setting only status first would leave a CANCELLED ticket with a seat and violate ck_res_seat_status immediately.
Project connection: UPDATE in cancel_ticket.

Q5. How is the waitlist order decided?
Short answer: Earliest booking_date, then lowest PNR.
Detailed: ORDER BY booking_date, pnr with FETCH FIRST 1 ROW ONLY gives FIFO order.
Project connection: PNR 4200001009 promoted.

Q6. What refund does a waitlisted ticket get?
Short answer: 100%.
Detailed: The first CASE branch; pay_status becomes REFUNDED because refund >= amount.
Project connection: test suite 04 step 4.

Q52 · Demo reset and new journey 130 (6.3)

HOW TO EXPLAIN THIS TO THE EXAMINER

"Before the demo we delete any old data for journey 130 — children first: cancellations, payments, reservations, then the schedule — and recreate it seven days from today, so the demo works the same whenever it is run."

Q1. Why delete children before the parent?
Short answer: Foreign keys.
Detailed: Deleting a parent with remaining children fails with ORA-02292 (unless cascades exist).
Project connection: order of the four DELETEs.

Q2. Why TRUNC(SYSDATE) + 7?
Short answer: A future date without a time part.
Detailed: Seven days ahead puts a cancellation in the 90% tier; TRUNC satisfies the date CHECK.
Project connection: journey 130.

Q3. Are the cascades needed for this to work?
Short answer: No.
Detailed: The explicit DELETEs remove children first; the cascades would do the same if the first two DELETEs were removed.
Project connection: fk_pay_res, fk_cancel_res.

Q53 · Demo: nine bookings in a loop (6.3)

HOW TO EXPLAIN THIS TO THE EXAMINER

"This anonymous block calls book_ticket nine times for an 8-seat EC coach. The first eight get seats 3001 to 3008 and the ninth is placed on the waitlist, showing the seat allocation and waitlist logic."

Q1. Must the FOR loop variable p be declared?
Short answer: No.
Detailed: Numeric FOR loop indexes are declared implicitly and exist only inside the loop.
Project connection: `FOR p IN 11 .. 19`.

Q2. Why does passenger 19 get WAITLIST?
Short answer: All 8 EC seats are taken.
Detailed: The free-seat query finds no row → NO_DATA_FOUND → v_seat NULL.
Project connection: Q50.

Q3. How do you call a procedure with OUT parameters?
Short answer: Pass variables in those positions.
Detailed: The procedure assigns them; after the call they hold the results.
Project connection: v_pnr, v_status.

Q54 · Demo: cancel and auto-promote (6.4)

HOW TO EXPLAIN THIS TO THE EXAMINER

"This block looks up passenger 11's ticket on journey 130 and cancels it. Seven days before departure the refund is 90 percent, 1588.50, and the freed seat 3001 is given automatically to the waitlisted passenger 19."

Q1. Why is the refund 1588.5?
Short answer: 90% of 1765.
Detailed: More than 48 hours remain before departure.
Project connection: refund CASE.

Q2. Which seat does the promoted passenger get?
Short answer: 3001, the seat just freed.
Detailed: cancel_ticket passes v_seat to the promotion UPDATE.
Project connection: E1-1 in Q55.

Q3. What happens if you run this block twice?
Short answer: ORA-20012 on the second run.
Detailed: The ticket is already CANCELLED.
Project connection: cancel_ticket status check.

Q55 · Demo result: six-table join (6.4)

HOW TO EXPLAIN THIS TO THE EXAMINER

"The final query joins six tables to show every ticket of journey 130. Seat, coach and cancellation are LEFT JOINs, so the cancelled ticket without a seat still appears and non-cancelled tickets show no refund. It confirms passenger 11 cancelled with a 1588.5 refund and the waitlisted passenger now holds seat E1-1."

Q1. Why must the coach join also be a LEFT JOIN?
Short answer: Otherwise it removes the row the seat LEFT JOIN kept.
Detailed: For the cancelled ticket st.coach_id is NULL; an inner join to coach finds no match and drops the row.
Project connection: PNR 4200001001.

Q2. Why does the cancelled ticket show '-' as the seat?
Short answer: Oracle treats NULL as an empty string in `||`.
Detailed: NULL || '-' || NULL = '-'.
Project connection: seat column.

Q3. How many rows would an inner join on cancellation return?
Short answer: One.
Detailed: Only PNR 4200001001 has a cancellation row.
Project connection: LEFT JOIN cancellation.

6. File 04_queries.sql — Views, Analysis Queries and the Capacity-Risk Module

Q56 · VIEW v_reservation_performance

HOW TO EXPLAIN THIS TO THE EXAMINER

"v_reservation_performance gives one row per journey. It joins schedules to trains, to a pre-aggregated capacity subquery, and LEFT JOINs to reservations and cancellations so journeys with no tickets still appear. Conditional aggregation with COUNT(CASE…) counts occupied, waitlisted and cancelled tickets in one pass, and we compute occupancy, cancellation rate with NULLIF to avoid division by zero, and revenue including cancellation charges retained. Other views and reports reuse it instead of repeating the logic."

Q1. What is a view and does it store data?
Short answer: A stored query; it stores no data.
Detailed: Querying the view runs its SELECT against the current tables. (A *materialized view* stores results; not used here.)
Project connection: three views in this file.

Q2. Why COUNT(r.pnr) and not COUNT(*)?
Short answer: To get 0 for journeys with no tickets.
Detailed: With LEFT JOIN, a journey without tickets produces one row with NULL ticket columns. COUNT(*) counts that row; COUNT(r.pnr) ignores NULLs.
Project connection: total_bookings.

Q3. Why is capacity calculated in a subquery?
Short answer: To avoid multiplying rows.
Detailed: Joining coaches and tickets together creates every coach × ticket combination; aggregating coaches first gives one capacity per train.
Project connection: inline view `cap`.

Q4. What does NULLIF do here?
Short answer: Prevents division by zero.
Detailed: NULLIF(COUNT(r.pnr), 0) is NULL when there are no tickets; dividing by NULL gives NULL instead of an error.
Project connection: cancellation_rate.

Q5. Can you INSERT into this view?
Short answer: No.
Detailed: Views with GROUP BY, aggregates or joins to aggregated subqueries are not updatable; Oracle cannot map a new row back to base tables. An INSTEAD OF trigger would be required.
Project connection: read-only reporting view.

Q57 · VIEW v_train_utilization

HOW TO EXPLAIN THIS TO THE EXAMINER

"v_train_utilization summarises the journey view per train. It joins STATION twice with different aliases to build a route label, sums capacity, occupied, waitlisted and cancelled across journeys, and recomputes the percentages as ratios of sums so bigger journeys weigh more. Demand percent includes waitlisted passengers, so it can go above 100."

Q1. Why join STATION twice?
Short answer: Origin and destination are different stations.
Detailed: Each join needs its own alias so both cities can be selected.
Project connection: `so`, `sd`.

Q2. Ratio of sums vs average of ratios?
Short answer: Ratio of sums weights by size.
Detailed: (10/10 + 0/100)/2 = 50% vs 10/110 = 9%. The second is the true occupancy.
Project connection: occupancy_pct.

Q3. How can demand exceed 100%?
Short answer: Waitlisted passengers are added to occupied seats.
Detailed: 12639: (93 + 7)/96 = 104.17%.
Project connection: demand_pct used by the risk module.

Q58 · VIEW v_ticket_details

HOW TO EXPLAIN THIS TO THE EXAMINER

"v_ticket_details is a de-normalised ticket view. It joins ROUTE_STOP twice — once for the boarding station and once for the destination, each on route and station — to compute the distance travelled, and uses LEFT JOINs to seat and coach so waitlisted and cancelled tickets still appear."

Q1. Why two conditions in the ROUTE_STOP join?
Short answer: A station can be on many routes.
Detailed: route_id + station_code identifies one stop (it is a unique key).
Project connection: uq_rs_station.

Q2. Is a de-normalised view bad design?
Short answer: No; the tables stay normalised.
Detailed: The view only presents joined data; it stores nothing, so no redundancy or anomalies.
Project connection: v_ticket_details.

Q3. Which query uses this view?
Short answer: Q8 (passenger travel frequency).
Detailed: It sums distance_km and fare per passenger.
Project connection: Q70.

Q59 · Querying the two summary views

HOW TO EXPLAIN THIS TO THE EXAMINER

"We query the views like tables: the journey view sorted by train and date, and the train view sorted by occupancy descending."

Q1. Is ORDER BY allowed inside a view definition?
Short answer: Allowed but not reliable.
Detailed: Order should be applied by the query that reads the view.
Project connection: views have no ORDER BY.

Q2. What is the default sort direction?
Short answer: ASC.
Detailed: NULLs sort last in ascending order in Oracle.
Project connection: train_no.

Q3. How are ties ordered?
Short answer: Unpredictably.
Detailed: Add a second sort key for a stable order.
Project connection: 12639 and 12027.

Q60 · Q1 Train-wise reservation statistics

HOW TO EXPLAIN THIS TO THE EXAMINER

"Q1 joins tickets to journeys and trains, groups by train, and uses conditional aggregation — COUNT of a CASE expression — to pivot the status counts into columns in a single pass. It also shows average fare and first and last booking dates, sorted by number of bookings."

Q1. What is conditional aggregation?
Short answer: An aggregate over a CASE expression.
Detailed: CASE returns a value only for rows meeting a condition; COUNT/SUM then counts or sums only those rows.
Project connection: status columns.

Q2. Why can ORDER BY use `bookings` but WHERE cannot?
Short answer: Logical order.
Detailed: WHERE runs before SELECT creates aliases; ORDER BY runs after.
Project connection: `ORDER BY bookings DESC`.

Q3. Why GROUP BY train_name as well as train_no?
Short answer: Every non-aggregated SELECT column must be grouped.
Detailed: train_name depends on train_no, so it doesn't change the groups.
Project connection: GROUP BY t.train_no, t.train_name.

Q61 · Q2 Station-wise passenger volume

HOW TO EXPLAIN THIS TO THE EXAMINER

"Q2 starts from all stations and LEFT JOINs two pre-aggregated subqueries — boardings grouped by from_station and alightings grouped by to_station — so every station appears, even TVC with zero traffic. NVL converts the missing counts to zero before adding them."

Q1. Why LEFT JOIN?
Short answer: Keep stations with no traffic.
Detailed: An inner join drops stations missing from either subquery.
Project connection: TVC, MAS alighted = 0.

Q2. What is 557 + NULL?
Short answer: NULL.
Detailed: Arithmetic with NULL gives NULL; NVL fixes it.
Project connection: total_footfall.

Q3. Why aggregate inside subqueries?
Short answer: Avoid multiplying rows.
Detailed: Two independent 1:N joins on the same parent create a cross product per station.
Project connection: subqueries b and a.

Q62 · Q3 Route utilization

HOW TO EXPLAIN THIS TO THE EXAMINER

"Q3 rolls the train view up to routes: number of trains, journeys, seats offered and sold, and occupancy as a ratio of sums. Chennai–Bengaluru is the fullest at 85 percent."

Q1. Why is distance_km in GROUP BY?
Short answer: It is selected and not aggregated.
Detailed: It depends on the route, so grouping by it doesn't split groups.
Project connection: GROUP BY ro.route_name, ro.distance_km.

Q2. Why does Q3's revenue differ from Q9's?
Short answer: Different definitions.
Detailed: Q3 counts travelled fares + retained cancellation charges; Q9 counts money collected minus refunds, which includes unrefunded waitlist payments on upcoming journeys.
Project connection: ₹1,015 / ₹400 differences.

Q3. What does COUNT(*) count here?
Short answer: Trains on the route.
Detailed: Each row of v_train_utilization is one train.
Project connection: trains column.

Q63 · Q4 Coach occupancy

HOW TO EXPLAIN THIS TO THE EXAMINER

"Q4 computes each coach's occupancy as seats sold divided by seats times journeys. It LEFT JOINs seats and reservations with the status condition inside the ON clause, so unsold seats are kept — putting that condition in WHERE would silently turn it into an inner join."

Q1. ON vs WHERE in a LEFT JOIN?
Short answer: ON filters what matches; WHERE filters the final rows.
Detailed: A WHERE condition on the right table removes the NULL-extended rows, converting the outer join to an inner join.
Project connection: status filter in Q4.

Q2. What is a seat-journey?
Short answer: One seat on one journey.
Detailed: seats_per_coach × journeys = total sellable units.
Project connection: seat_journeys column.

Q3. Why join the journeys subquery?
Short answer: To know how many times each seat could be sold.
Detailed: Pre-aggregated to avoid multiplying seat rows.
Project connection: subquery j.

Q64 · Q5a Cancellation statistics by train

HOW TO EXPLAIN THIS TO THE EXAMINER

"Q5a LEFT JOINs tickets to cancellations, so COUNT of PNR gives all bookings and COUNT of cancel_id gives only cancellations. It sums refunds and retained charges, and averages the days between cancellation and journey — AVG ignores the NULLs of non-cancelled tickets. Chennai Shatabdi has the highest rate at 12.5 percent."

Q1. COUNT(*) vs COUNT(column)?
Short answer: COUNT(column) skips NULLs.
Detailed: After a LEFT JOIN, COUNT(cn.cancel_id) counts only matched rows.
Project connection: cancellations column.

Q2. How does AVG treat NULLs?
Short answer: Ignores them.
Detailed: AVG = SUM(non-NULL)/COUNT(non-NULL).
Project connection: avg_days_before_journey.

Q3. What are charges retained?
Short answer: Fare minus refund on cancelled tickets.
Detailed: Money the railway keeps as cancellation fees.
Project connection: 773.2 for 12027.

Q65 · Q5b Cancellation reasons

HOW TO EXPLAIN THIS TO THE EXAMINER

"Q5b groups cancellations by reason and uses the analytic function RATIO_TO_REPORT over the whole result to give each reason's share. Window functions are evaluated after GROUP BY, which is why they can use COUNT(*) as input."

Q1. What does OVER () mean?
Short answer: The window is the whole result.
Detailed: No partition or ordering — all rows form one window.
Project connection: share_pct.

Q2. Can a window function use an aggregate?
Short answer: Yes.
Detailed: Window functions run after GROUP BY/HAVING.
Project connection: RATIO_TO_REPORT(COUNT(*)).

Q3. ANSI alternative?
Short answer: COUNT(*) / SUM(COUNT(*)) OVER ().
Detailed: Works in most databases.
Project connection: Q5b.

Q66 · Q6a Peak travel routes (DENSE_RANK)

HOW TO EXPLAIN THIS TO THE EXAMINER

"Q6a counts passengers per route, including waitlisted ones because they show demand, and ranks routes with DENSE_RANK over the count. DENSE_RANK gives ties the same rank without gaps."

Q1. RANK vs DENSE_RANK vs ROW_NUMBER?
Short answer: Gaps / no gaps / always unique.
Detailed: Scores 100, 100, 90 → RANK 1,1,3; DENSE_RANK 1,1,2; ROW_NUMBER 1,2,3.
Project connection: demand_rank.

Q2. Is the output sorted?
Short answer: Not guaranteed.
Detailed: The window's ORDER BY decides ranks only; the result needs its own ORDER BY.
Project connection: missing ORDER BY.

Q3. Why include WAITLIST?
Short answer: It measures demand.
Detailed: People who wanted to travel but could not are part of demand.
Project connection: compare A4 (status <> 'CANCELLED', same set).

Q67 · Q6b Top 5 origin–destination pairs

HOW TO EXPLAIN THIS TO THE EXAMINER

"Q6b joins STATION twice to get boarding and destination cities, counts passengers per city pair, sorts descending and keeps the first five with FETCH FIRST. It used to use ROWNUM in a subquery, because ROWNUM is applied before ORDER BY."

Q1. Why can't you write `WHERE ROWNUM <= 5 ORDER BY …` in one query?
Short answer: ROWNUM is assigned before sorting.
Detailed: You'd get 5 random rows, then sorted.
Project connection: the original Q6b used a subquery.

Q2. What does FETCH FIRST do with ties?
Short answer: Ignores them unless WITH TIES.
Detailed: ONLY returns exactly N rows.
Project connection: top 5 segments.

Q3. Why group by fs.city, ts.city instead of the alias?
Short answer: GROUP BY runs before SELECT.
Detailed: The alias does not exist yet (Oracle 23ai added alias support in GROUP BY, but the portable form uses columns).
Project connection: segment.

Q68 · Q6c Peak travel days of the week

HOW TO EXPLAIN THIS TO THE EXAMINER

"Q6c groups travelled tickets by the day name of the journey date using TO_CHAR with the 'DY' format, and shows each day's share with RATIO_TO_REPORT. Sunday is the busiest day in our data."

Q1. Can you GROUP BY an expression?
Short answer: Yes.
Detailed: The SELECT must use the same expression (or aggregates).
Project connection: TO_CHAR(journey_date,'DY').

Q2. Why might day names differ on another machine?
Short answer: NLS_DATE_LANGUAGE.
Detailed: TO_CHAR uses the session's language unless specified.
Project connection: travel_day.

Q3. Is the result sorted by weekday?
Short answer: No, by passengers.
Detailed: `ORDER BY passengers DESC`.
Project connection: Q6c.

Q69 · Q7 Highest and lowest utilized trains

HOW TO EXPLAIN THIS TO THE EXAMINER

"Q7 ranks trains by occupancy twice, descending and ascending, in an inline view, and the outer query keeps rank 1 of either. We need the outer query because window functions can't be used in WHERE. RANK returns both 12639 and 12027 since they tie at 96.88 percent."

Q1. Why is an inline view needed?
Short answer: WHERE runs before window functions.
Detailed: Compute the rank in the inner query, filter in the outer.
Project connection: rk_high, rk_low.

Q2. Why three rows instead of two?
Short answer: A tie for highest.
Detailed: RANK gives both 96.88% trains rank 1.
Project connection: 12639, 12027.

Q3. Alternative without window functions?
Short answer: Compare with MAX and MIN subqueries.
Detailed: Equivalent result including ties.
Project connection: Q7.

Q70 · Q8 Passenger travel frequency

HOW TO EXPLAIN THIS TO THE EXAMINER

"Q8 joins the ticket-details view to passengers, keeps confirmed and completed trips, and per passenger counts trips, sums distance and fare, and uses a CASE on COUNT(*) to classify them as frequent, regular or occasional. The CASE order matters because the first true branch wins."

Q1. Can CASE use an aggregate?
Short answer: Yes, in a grouped query.
Detailed: It is evaluated once per group after aggregation.
Project connection: traveller_type.

Q2. Why is the CASE order important?
Short answer: The first true WHEN wins.
Detailed: Check the stricter condition first.
Project connection: `>= 6` before `>= 3`.

Q3. Where does distance come from?
Short answer: v_ticket_details.
Detailed: The view subtracts stop distances.
Project connection: Q58.

Q71 · Q9 Revenue by train with ROLLUP

HOW TO EXPLAIN THIS TO THE EXAMINER

"Q9 computes per payment the amount and the refund — either from the cancellation table or a full refund of a waitlisted ticket — in an inline view, then GROUP BY ROLLUP on route and train gives train rows, a subtotal per route and a grand total in one query. GROUPING() tells us which rows are subtotals so we can label them, and the ORDER BY puts details before subtotals and the grand total last."

Q1. What does ROLLUP(a, b) produce?
Short answer: Groups (a,b), (a), and ().
Detailed: n columns give n+1 grouping levels.
Project connection: train rows, route totals, grand total.

Q2. What does GROUPING(col) return?
Short answer: 1 on rows where col was aggregated away, else 0.
Detailed: Distinguishes subtotal NULLs from real NULLs.
Project connection: labels and ORDER BY.

Q3. ROLLUP vs CUBE?
Short answer: ROLLUP is hierarchical; CUBE gives all combinations.
Detailed: CUBE(a,b) → (a,b), (a), (b), ().
Project connection: ROLLUP fits route → train.

Q4. Why did `ORDER BY GROUPING(route_name)` fail?
Short answer: It resolved to the SELECT alias, an expression.
Detailed: ORDER BY prefers select-list aliases; GROUPING needs a GROUP BY column, so qualify it with the table alias.
Project connection: x.route_name.

Q72 · Q10 Routes requiring additional capacity (HAVING)

HOW TO EXPLAIN THIS TO THE EXAMINER

"Q10 aggregates the journey view by route and uses HAVING to keep routes that both had waitlisted passengers and overall occupancy of at least 75 percent. HAVING is needed because these are conditions on aggregates; WHERE only filters individual rows before grouping."

Q1. WHERE vs HAVING?
Short answer: WHERE filters rows before grouping; HAVING filters groups after.
Detailed: Aggregates are only allowed in HAVING (and SELECT/ORDER BY).
Project connection: Q10.

Q2. Why repeat the occupancy expression in HAVING?
Short answer: Aliases don't exist yet.
Detailed: HAVING runs before SELECT computes aliases.
Project connection: occupancy_pct.

Q3. Which routes qualify?
Short answer: Chennai–Bengaluru and Chennai–Madurai.
Detailed: Both have waitlisted passengers and ≥ 75% occupancy.
Project connection: output.

Q73 · A1 Ten-table join: a passenger's travel history

HOW TO EXPLAIN THIS TO THE EXAMINER

"A1 builds a passenger's complete travel history by joining ten table references along their foreign keys. STATION is joined twice with different aliases for boarding and destination, and SEAT and COACH are LEFT JOINs so tickets without a seat would still appear."

Q1. What causes a Cartesian product?
Short answer: A missing or always-true join condition.
Detailed: Every row of one table pairs with every row of the other.
Project connection: station joins.

Q2. Why aliases fs and ts?
Short answer: STATION appears twice.
Detailed: Aliases tell Oracle which copy each column comes from.
Project connection: boarding, destination.

Q3. Why is payment an inner join?
Short answer: Every ticket has exactly one payment.
Detailed: The 1:1 relationship guarantees a match.
Project connection: uq_pay_pnr.

Q74 · A2 Correlated subquery: trains above their route's average

HOW TO EXPLAIN THIS TO THE EXAMINER

"A2 uses a correlated subquery: for each train, the inner query averages the occupancy of trains with the same route_id as the outer row, and the outer query keeps the train if it is above that average. The same subquery in the SELECT list shows the route average."

Q1. What makes a subquery correlated?
Short answer: It references the outer query.
Detailed: `x.route_id = u.route_id` uses u from outside.
Project connection: A2.

Q2. Is a correlated subquery always slow?
Short answer: Not necessarily.
Detailed: Oracle can unnest it into a join or cache results; check the execution plan.
Project connection: small data here.

Q3. Why is 12677 not listed?
Short answer: It is the only train on its route.
Detailed: Its value equals the average, and `>` is strict.
Project connection: route 4.

Q75 · A3 Passengers with repeated cancellations

HOW TO EXPLAIN THIS TO THE EXAMINER

"A3 groups cancelled tickets by passenger, keeps those with two or more using HAVING, uses a correlated scalar subquery to count each passenger's total bookings for the ratio, and LISTAGG to list the cancelled trains in date order."

Q1. What does LISTAGG do?
Short answer: Concatenates values of a group into one string.
Detailed: WITHIN GROUP (ORDER BY …) sets the order.
Project connection: trains_cancelled.

Q2. Why a subquery for total bookings?
Short answer: The WHERE removed non-cancelled tickets.
Detailed: The group contains only cancelled rows.
Project connection: total_bookings.

Q3. Can the subquery use p.passenger_id?
Short answer: Yes, it is a GROUP BY column.
Detailed: Correlated references in grouped queries must be grouped columns.
Project connection: A3.

Q76 · A4 Nested subquery: routes above average demand

HOW TO EXPLAIN THIS TO THE EXAMINER

"A4 counts passengers per route and keeps routes above the average route count. The average is computed by a nested non-correlated subquery: the inner query counts per route and the middle one averages those counts, which here is 185.25."

Q1. Is this subquery correlated?
Short answer: No.
Detailed: It does not reference the outer query; it runs once.
Project connection: A4.

Q2. Why HAVING, not WHERE?
Short answer: It compares an aggregate.
Detailed: COUNT(*) per route only exists after grouping.
Project connection: A4.

Q3. What is the average used?
Short answer: 185.25.
Detailed: (267 + 266 + 157 + 51) / 4.
Project connection: output.

Q77 · A5 WITH clause: stations above average traffic

HOW TO EXPLAIN THIS TO THE EXAMINER

"A5 defines a footfall CTE that stacks boarding and alighting station codes with UNION ALL and counts movements per station. The main query joins it to STATION and keeps stations above the average, computed by a scalar subquery on the same CTE: 110.9."

Q1. What is a CTE?
Short answer: A named subquery defined with WITH.
Detailed: Improves readability; can be referenced multiple times.
Project connection: footfall.

Q2. Why UNION ALL?
Short answer: Every movement must be counted.
Detailed: UNION would remove duplicate station codes.
Project connection: footfall.

Q3. Why is the average 110.9 and not 103?
Short answer: TVC (0 movements) is not in the CTE.
Detailed: 1442/13 vs 1442/14.
Project connection: compare Q2.

Q78 · A6 NOT EXISTS + EXISTS: trains needing capacity

HOW TO EXPLAIN THIS TO THE EXAMINER

"A6 expresses 'every journey was at least 90 percent full' as 'there does not exist a journey below 90 percent' using a correlated NOT EXISTS, and adds an EXISTS check that someone was waitlisted. That returns 12639, 12027 and 12637."

Q1. How do you express "for all" in SQL?
Short answer: NOT EXISTS of a counter-example.
Detailed: "All journeys ≥ 90" ≡ "no journey < 90".
Project connection: A6.

Q2. Why SELECT 1 inside EXISTS?
Short answer: Only row existence matters.
Detailed: Any select list works; 1 is a convention.
Project connection: A6.

Q3. EXISTS vs IN?
Short answer: EXISTS tests for rows; IN compares with a list.
Detailed: NOT EXISTS is safe with NULLs; NOT IN returns no rows if the list contains NULL.
Project connection: compare A8.

Q79 · A7 Correlated HAVING: above-average spenders per city

HOW TO EXPLAIN THIS TO THE EXAMINER

"A7 totals each passenger's spending and, in HAVING, compares it with a correlated subquery that computes the average spend per traveller in the same home city — total city spend divided by COUNT(DISTINCT passenger_id). Passengers above their city's average are listed."

Q1. What does COUNT(DISTINCT col) do?
Short answer: Counts unique non-NULL values.
Detailed: Here: distinct travelling passengers per city.
Project connection: A7.

Q2. Can HAVING contain a subquery?
Short answer: Yes.
Detailed: Including correlated ones referencing grouped columns.
Project connection: A7, A4.

Q3. What happens for a NULL home city?
Short answer: Excluded.
Detailed: NULL = NULL is UNKNOWN.
Project connection: three-valued logic.

Q80 · A8 IN and NOT IN: loyal passengers

HOW TO EXPLAIN THIS TO THE EXAMINER

"A8 uses IN with a subquery that finds passengers with at least three trips on trains averaging 90 percent occupancy — itself using another IN — and NOT IN to exclude anyone who ever cancelled. NOT IN is safe here because passenger_id is NOT NULL; otherwise a NULL in the list would make it return nothing."

Q1. Why can NOT IN return no rows?
Short answer: A NULL in the list makes every comparison UNKNOWN.
Detailed: x NOT IN (1, NULL) = x<>1 AND x<>NULL → UNKNOWN.
Project connection: safe because passenger_id is NOT NULL.

Q2. IN vs JOIN?
Short answer: IN doesn't duplicate outer rows.
Detailed: A join with a 1:N subquery result would repeat passengers.
Project connection: A8.

Q3. Which trains are "near-full"?
Short answer: 12639, 12027, 12637.
Detailed: Occupancy ≥ 90% in v_train_utilization.
Project connection: V2 output.

Q81 · A9 Correlated subquery on a CTE: under-used coaches

HOW TO EXPLAIN THIS TO THE EXAMINER

"A9 defines coach occupancy in a WITH clause, then for each coach uses a correlated subquery on that CTE to get its own train's average coach occupancy, and lists coaches more than 10 points below it."

Q1. Can a CTE be used in a correlated subquery?
Short answer: Yes.
Detailed: It behaves like a table within the statement.
Project connection: coach_occ x.

Q2. Why JOIN seat here but LEFT JOIN seat in Q4?
Short answer: No difference in this data.
Detailed: Every coach has seats; LEFT JOIN would matter only for a coach without seats.
Project connection: Q4 vs A9.

Q3. What does the -10 mean?
Short answer: A 10-percentage-point margin.
Detailed: Avoids flagging coaches only slightly below average.
Project connection: A9.

Q82 · CREATE TABLE risk_threshold and its rows

HOW TO EXPLAIN THIS TO THE EXAMINER

"RISK_THRESHOLD stores the classification limits as key-value rows, so the business can tune rules with an UPDATE instead of changing code. The function reads these values every time it classifies a train."

Q1. Why store thresholds in a table?
Short answer: Change policy without changing code.
Detailed: Supports what-if analysis and auditing of rule changes.
Project connection: Q87.

Q2. What if a threshold row is missing?
Short answer: The variable is NULL; that rule never matches.
Detailed: `x >= NULL` is UNKNOWN, which IF treats as false.
Project connection: fn_capacity_risk.

Q3. Drawback of key–value tables?
Short answer: No per-parameter typing or constraints; pivoting needed.
Detailed: All values share one column type and one CHECK.
Project connection: MAX(DECODE()) in Q84.

Q83 · CREATE TABLE capacity_risk_log

HOW TO EXPLAIN THIS TO THE EXAMINER

"CAPACITY_RISK_LOG keeps the daily classification of every train. Its primary key is run date plus train number, so each train is classified at most once a day and history builds up over time."

Q1. Why a composite PK with run_date?
Short answer: One row per train per day.
Detailed: Keeps history and makes daily re-runs idempotent.
Project connection: prc_capacity_risk_report.

Q2. Is the COMMIT needed?
Short answer: Not strictly.
Detailed: The CREATE TABLE already committed the INSERTs implicitly; the COMMIT makes the intent explicit.
Project connection: Q83.

Q3. Why store measures as well as the status?
Short answer: To explain and re-evaluate decisions later.
Detailed: The what-if analysis re-classifies stored measures with new thresholds.
Project connection: Q87.

Q84 · FUNCTION fn_capacity_risk

HOW TO EXPLAIN THIS TO THE EXAMINER

"fn_capacity_risk is a stored function, so we can call it inside SQL. It reads all five thresholds from RISK_THRESHOLD in one query, using MAX(DECODE(...)) to pivot rows into variables, then applies the rules with IF/ELSIF: capacity required if occupancy and demand are high and cancellations low; high demand if occupancy or demand is high; otherwise normal. Keeping the rules in one function means every report classifies trains the same way."

Q1. Function vs procedure?
Short answer: A function returns a value and can be used in SQL; a procedure is called as a statement.
Detailed: Functions called from SQL must not modify data (no DML/COMMIT in a query context).
Project connection: fn_capacity_risk vs prc_capacity_risk_report.

Q2. How does MAX(DECODE()) pivot rows?
Short answer: DECODE keeps one value per column, MAX collapses the rows.
Detailed: Each output column is non-NULL in exactly one input row.
Project connection: threshold read.

Q3. Why does 12027 get HIGH DEMAND, not CAPACITY REQUIRED?
Short answer: Its cancellation rate 12.5% exceeds 10%.
Detailed: Many cancellations free seats, so the waitlist may clear by itself.
Project connection: Q85, Q87.

Q4. Why is NVL applied to the cancel rate?
Short answer: A NULL rate (no bookings) is treated as 0.
Detailed: Avoids UNKNOWN comparisons.
Project connection: `NVL(p_cancel_rate, 0)`.

Q85 · 10.1 Decision-support report

HOW TO EXPLAIN THIS TO THE EXAMINER

"The decision-support report reads the train view, adds each route's average occupancy with a window function, calls fn_capacity_risk to classify each train, and uses a CASE on the result to recommend an action — for capacity-required trains it computes the extra seats per journey with CEIL of waitlisted over journeys. A DECODE in ORDER BY puts the most urgent category first. Here 12639 and 12637 need capacity."

Q1. Window AVG vs GROUP BY AVG?
Short answer: Window keeps every row; GROUP BY collapses rows.
Detailed: PARTITION BY defines the groups for the calculation only.
Project connection: "Route Avg Occ %".

Q2. Simple CASE vs searched CASE?
Short answer: Simple compares one expression to values; searched evaluates conditions.
Detailed: `CASE x WHEN 'A' THEN …` vs `CASE WHEN x < 50 THEN …`.
Project connection: both in the action column.

Q3. Why CEIL?
Short answer: You can't add part of a seat.
Detailed: Rounding up ensures enough seats for the average waitlist.
Project connection: 7/3 → 3.

Q4. What are quoted identifiers?
Short answer: Names in double quotes, case-sensitive, may contain spaces.
Detailed: "Occupancy %" must be referenced exactly so.
Project connection: report column headings.

Q86 · PROCEDURE prc_capacity_risk_report (10.2)

HOW TO EXPLAIN THIS TO THE EXAMINER

"prc_capacity_risk_report deletes today's log rows, inserts today's classification of every train using the view and the function, prints a summary with a cursor FOR loop over a grouped query with LISTAGG, and commits. Deleting today's rows first makes it safe to run several times a day."

Q1. What is EXEC?
Short answer: SQL*Plus shorthand for an anonymous block call.
Detailed: `EXEC p` = `BEGIN p; END;`.
Project connection: EXEC prc_capacity_risk_report.

Q2. Why DELETE before INSERT?
Short answer: Idempotency.
Detailed: Avoids duplicate-key errors on re-runs the same day.
Project connection: PK (run_date, train_no).

Q3. Risk of COMMIT inside a procedure?
Short answer: It commits the caller's work too.
Detailed: Transactions belong to the session, not the procedure (unless it is an autonomous transaction).
Project connection: compare book_ticket.

Q87 · 10.3 What-if analysis with ROLLBACK

HOW TO EXPLAIN THIS TO THE EXAMINER

"The what-if analysis updates the cancellation threshold to 15 percent inside a transaction, re-classifies the stored measures with the function — which sees our own uncommitted change — and lists the trains whose status would change, which is only 12027. Then ROLLBACK undoes the change, so nothing is permanent and other users never saw it."

Q1. Does another session see the 15% during the analysis?
Short answer: No.
Detailed: Uncommitted changes are visible only to their own session (isolation).
Project connection: Q87.

Q2. Why does the function see 15?
Short answer: Same session.
Detailed: A session always sees its own uncommitted changes.
Project connection: fn_capacity_risk reads risk_threshold.

Q3. What does ROLLBACK undo here?
Short answer: Only the UPDATE.
Detailed: Everything since the last commit (the procedure committed just before).
Project connection: 10.3.

7. Runner Script and SQL*Plus Commands

R1 · parts/run_all.sql — the master runner

HOW TO EXPLAIN THIS TO THE EXAMINER

"run_all.sql is a SQL*Plus script that sets the session — server output on, continue on errors because some tests are meant to fail — and runs the five part files in dependency order with @@, which resolves paths relative to the runner itself."

Q1. Is SET SERVEROUTPUT ON a SQL statement?
Short answer: No, a SQL*Plus client command.
Detailed: The database buffers DBMS**UTPUT; the client m**t ask to fetch and show it.
Project connection: run_all.sql.

Q2. @ vs @@?
Short answer: @@ is relative to the calling script's folder.
Detailed: @ is relative to the current directory of SQL*Plus.
Project connection: @@01_ddl.sql.

Q3. Why WHENEVER SQLERROR CONTINUE?
Short answer: Part 5 deliberately causes errors.
Detailed: The handlers catch most, but the clean-up must also survive any unexpected error.
Project connection: integrity tests.